

Banque MPI Par Points d'Evaluation / 按考察点分层的 MPI 题库

1. Environnement MPI et Notions de Base / 1. MPI 环境与基础概念

Definitions De Base

1.01 [Definition:MPI] Qu'est-ce que MPI ? / 什么是 MPI?

Reponse : MPI signifie Message Passing Interface . C'est une interface standard de communication entre processus pour les programmes paralleles a memoire distribuee, implementee par des bibliotheques comme MPICH ou Open MPI. / 答：MPI 是 Message Passing Interface，是分布式内存并行程序中进程间通信的标准接口，由 MPICH、Open MPI 等库实现。

Exemple complet : voir la question 1 de [APPPartielCorr2024.md](#).

1.02 [Definition:Init] Quel est le role de MPI_Init ? / MPI_Init 的作用是什么?

Reponse : MPI_Init initialise l'environnement MPI. Il doit etre appele avant tout autre appel MPI. / 答：MPI_Init 用于初始化 MPI 环境，必须在其他 MPI 调用之前执行。

Exemple complet : voir la question 2 de [APPPartielCorr2024.md](#).

1.03 [Definition:Finalize] Quel est le role de MPI_Finalize ? / MPI_Finalize 的作用是什么?

Reponse : MPI_Finalize termine proprement l'environnement MPI. Apres cet appel, il ne faut plus effectuer d'appel MPI. / 答：MPI_Finalize 用于正确结束 MPI 环境；调用后不能再进行 MPI 调用。

Exemple complet : voir la question 2 de [APPPartielCorr2024.md](#).

1.04 [Definition:Comm_size] Que renvoie MPI_Comm_size ? / MPI_Comm_size 返回什么?

Reponse : MPI_Comm_size ecrit dans la variable resultat le nombre total de processus appartenant au communicateur donne. / 答：MPI_Comm_size 把给定 communicator 中的进程总数写入结果变量。

Exemple complet : voir la question 2 de [APPPartielCorr2024.md](#).

1.05 [Definition:Comm_rank] Que renvoie MPI_Comm_rank ? / MPI_Comm_rank 返回什么?

Reponse : MPI_Comm_rank ecrit dans la variable resultat le rang du processus courant dans le communicateur donne. / 答：MPI_Comm_rank 把当前进程在给定 communicator 中的 rank 写入结果变量。

Exemple complet : voir la question 2 de [APPPartielCorr2024.md](#).

1.06 [Definition:COMM_WORLD] Que represente MPI_COMM_WORLD ? / MPI_COMM_WORLD 表示什么?

Reponse : MPI_COMM_WORLD est le communicateur par default contenant tous les processus MPI lances au depart. / 答：MPI_COMM_WORLD 是默认 communicator，包含程序启动时创建的全部 MPI 进程。

Exemple complet : voir la question 2 de [APPPartielCorr2024.md](#).

Processus Et Communicateurs

1.07 [Comparaison:Processus vs Thread] Quelle difference entre un processus MPI et un thread ? / MPI 进程和线程有什么区别?

Reponse : Un processus MPI possede son propre espace memoire et communique explicitement par appels MPI. Un thread partage en general la memoire du processus auquel il appartient. / 答：MPI 进程有自己独立的地址空间，通过 MPI 显式通信；线程通常共享所属进程的内存。

Exemple complet : voir la question 3.2 de [APP2023.md](#).

1.08 [Definition:Rang] Pourquoi dit-on que le rang est relatif a un communicateur ? / 为什么说 rank 是相对于 communicator 的?

Reponse : Parce que le rang n'est defini qu'a l'interieur d'un communicateur. Un meme processus peut donc avoir un rang dans MPI_COMM_WORLD et un autre rang dans un sous-communicateur. / 答：因为 rank 只在某个 communicator 内有意义；同一个进程在 MPI_COMM_WORLD 和子 communicator 中可以有不同 rank。

Exemple complet : voir la question 2 de [APPPartielCorr2024.md](#).

1.09 [Definition:Communicateur] Qu'est-ce qu'un communicateur MPI ? / 什么是 MPI communicator?

Reponse : Un communicateur MPI definit un groupe de processus et l'espace de communication dans lequel les rangs et les operations MPI sont interpretes. / 答：MPI communicator 定义了一组进程，以及 rank 和通信操作所处的通信空间。

Exemple complet : voir la question 2 de [APPPartielCorr2024.md](#).

Execution Et Logique Globale

1.10 [Question:Validite] Pourquoi aucun appel MPI ne doit-il etre fait avant MPI_Init ni apres MPI_Finalize ? / 为什么 MPI_Init 前和 MPI_Finalize 后不能调用 MPI?

Reponse : Avant MPI_Init , l'environnement MPI n'est pas encore initialise. Apres MPI_Finalize , il est termine. Dans les deux cas, les appels MPI ne sont donc pas valides. / 答：MPI_Init 前 MPI 运行环境尚未建立，MPI_Finalize 后环境已结束，因此这两个区间都不能合法调用 MPI。

Exemple complet : voir la question 2 de [APPPartielCorr2024.md](#).

1.11 [Question:Lancement] Comment lancer un programme MPI avec 4 processus ? / 如何用 4 个进程启动 MPI 程序?

Reponse : On peut obtenir un blocage, une attente infinie, ou plus generalement un programme incorrect, car certaines communications ou operations collectives exigent une participation coherente des processus concernes. / 答：可写作 mpirun -np 4 ./a.out ；根据实现不同，也常见 mpiexec -np 4 ./a.out 。

1.12 [Impl:Distribution] Donner un exemple d'utilisation de rang pour distribuer un travail. / 举一个利用 rank 分配工作的例子。

```
#include <mpi.h>
#include <stdio.h>
int main(int argc, char **argv) {
    int rang, P, N = 16;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rang);
    MPI_Comm_size(MPI_COMM_WORLD, &P);
    int debut = rang * N / P, fin = ((rang + 1) * N / P) - 1;
    printf("rang %d traite les indices [%d, %d]\n", rang, debut, fin);
    MPI_Finalize();
    return 0;
}
```

Reponse : Dans cet exemple, chaque processus calcule son intervalle local a partir de rang et de P, puis affiche la tranche dont il est responsable. / 答：在这个例子中，每个进程根据 rang 和 P 计算自己的局部区间，并输出自己负责的索引范围。

Types De Communications

1.13 [Question:Logique globale] Que peut-il se passer si un seul processus entre dans une phase MPI alors que les autres ne suivent pas la meme logique ? / 如果只有部分进程进入某个 MPI 阶段会发生什么问题?

Reponse : On peut obtenir un blocage, une attente infinie, ou plus generalement un programme incorrect, car certaines communications ou operations collectives exigent une participation coherente des processus concernes. / 答：这可能导致死锁、无限等待或整体逻辑错误，因为某些通信和集合操作要求相关进程按一致逻辑参与。

1.14 [Definition:Point-a-point] Qu'appelle-t-on une communication point-a-point ? / 什么叫点对点通信?

Reponse : Une communication point-a-point est une communication entre deux processus : un emetteur et un recepteur. / 答：点对点通信就是两个进程之间的通信，一个发送方，一个接收方。

Exemple complet : voir 2.07 .

1.15 [Definition:Collective] Qu'appelle-t-on une communication collective ? / 什么叫集合通信?

Reponse : Une communication collective est une communication impliquant l'ensemble des processus d'un meme communicateur, par exemple pour diffuser, rassembler, redistribuer ou reduire des donnees. / 答：集合通信是同一个 communicator 中全部进程共同参与的一类通信，例如用于广播、收集、重分发或归约数据。

1.16 [Comparaison:CP2P vs Collective] Quelle difference entre communication point-a-point et communication collective ? / 点对点通信和集合通信有什么区别?

Reponse : Une communication point-a-point relie explicitement un emetteur et un recepteur. Une communication collective implique au contraire tous les processus d'un meme communicateur selon une operation commune comme `Bcast` , `Gather` ou `Reduce` . / 答：点对点通信是发送方和接收方之间的显式通信；集合通信则是在同一个 communicator 中由所有相关进程共同参与的一种统一操作，例如 `Bcast` 、 `Gather` 或 `Reduce` 。

Exemple complet : voir 2.07 .

2. Communications Point-a-Point / 2. 点对点通信

Definitions Point-A-Point

2.01 [Comparaison:Send/Recv] Quelle difference entre `MPI_Send` et `MPI_Recv` ? / `MPI_Send` 和 `MPI_Recv` 有什么区别？

Reponse : `MPI_Send` est utilise par le processus emetteur pour envoyer des donnees. `MPI_Recv` est utilise par le processus recepteur pour recevoir des donnees dans un tampon de reception. / 答： `MPI_Send` 由发送进程调用来发送数据， `MPI_Recv` 由接收进程调用把数据接收到接收缓冲区中。

Exemple complet : voir 2.07 .

2.02 [Definition:Blocking] Qu'est-ce qu'une communication bloquante ? / 什么是阻塞通信？

Reponse : Dans une communication bloquante, l'appel ne se termine que lorsque l'operation de communication est terminee du point de vue local. Pour un envoi, cela signifie que le tampon d'envoi peut etre reutilise ; pour une reception, les donnees sont deja dans le tampon de reception. / 答：阻塞通信是指调用在本地完成前不会返回。对发送来说，返回时发送缓冲区已可安全重用；对接收来说，返回时数据已经实际写入接收缓冲区。

Exemple complet : voir 2.23 .

2.03 [Definition:NonBlocking] Qu'est-ce qu'une communication non bloquante ? / 什么是非阻塞通信？

Reponse : Dans une communication non bloquante, l'appel retourne sans attendre la fin effective de l'operation. La communication est seulement demarree et doit ensuite etre terminee explicitement avec `MPI_Wait` ou testee avec `MPI_Test` . / 答：非阻塞通信是指调用在通信真正完成前就返回；通信只是被启动，之后要用 `MPI_Wait` 显式等待完成或用 `MPI_Test` 检查。

Exemple complet : voir 2.10 .

2.04 [Definition:Status] A quoi sert `MPI_Status` ? / `MPI_Status` 有什么作用？

Reponse : `MPI_Status` permet de recuperer des informations sur le message effectivement reçu, notamment la vraie source, le vrai tag et un code d'erreur. Il est particulierement utile avec `MPI_ANY_SOURCE` ou `MPI_ANY_TAG` . / 答： `MPI_Status` 用于得到实际接收到的消息信息，尤其是真实 source、真实 tag 和错误码；在使用 `MPI_ANY_SOURCE` 或 `MPI_ANY_TAG` 时特别有用。

Exemple complet : voir 2.13 .

2.05 [Definition:Tag] A quoi sert un `tag` ? / `tag` 有什么作用？

Reponse : Le `tag` sert a etiqueter les messages afin de distinguer plusieurs types de communications entre les memes processus. / 答： `tag` 用来给消息打标签，以区分同一对进程之间不同类型的通信。

Exemple complet : voir 2.07 .

2.06 [Definition:Wait/Test] A quoi servent `MPI_Wait` et `MPI_Test` ? / `MPI_Wait` 和 `MPI_Test` 有什么作用？

Reponse : `MPI_Wait` attend la fin d'une communication non bloquante. `MPI_Test` verifie si cette communication est terminee sans bloquer. / 答： `MPI_Wait` 用于等待一次非阻塞通信完成； `MPI_Test` 用于在不阻塞的情况下检查它是否已完成。

Exemple complet : voir 2.21 et 2.24 .

Echanges Simples Et Anneau

2.07 [Impl:Echange 0<->1] Ecrire un echange bidirectionnel entre 0 et 1 sans interblocage. / 写一个 0 和 1 的双向交换且不死锁。

```
#include <mpi.h>
#include <stdio.h>
int main(int argc, char **argv) {
    int rang, x, y = -1; MPI_Request req1, req2;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rang);
    x = rang;
    if (rang == 0 || rang == 1) {
        int autre = 1 - rang;
        MPI_Isend(&x, 1, MPI_INT, autre, 0, MPI_COMM_WORLD, &req1);
        MPI_Irecv(&y, 1, MPI_INT, autre, 0, MPI_COMM_WORLD, &req2);
        MPI_Wait(&req1, MPI_STATUS_IGNORE);
        MPI_Wait(&req2, MPI_STATUS_IGNORE);
        printf("rang %d a reçu %d de %d\n", rang, y, autre);
    }
    MPI_Finalize();
    return 0;
}
```

2.08 [Debug:Deadlock] Pourquoi un schema `Send` puis `Recv` des deux cotes peut-il bloquer ? / 为什么双方都先 `Send` 再 `Recv` 可能死锁？

Reponse : Avec deux `MPI_Send` bloquants postes symetriquement, chaque processus peut attendre que l'autre poste sa reception. Le programme peut alors se bloquer, surtout si l'implementation utilise un protocole synchrone pour ce message. / 答：若双方都先做阻塞 `MPI_Send` ，每边都可能在等待对方先贴出接收，程序就可能死锁，尤其当该消息走同步协议时。

Exemple complet : voir 2.18 .

2.09 [Impl:Anneau] Concevoir un anneau complet de `P` etapes. / 设计一个完整 `P` 步环形传递。

```
#include <mpi.h>
#include <stdio.h>
int main(int argc, char **argv) {
    int rang, P, val, tmp, gauche, droite; MPI_Request req1, req2;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rang);
    MPI_Comm_size(MPI_COMM_WORLD, &P);
    gauche = (rang + P - 1) % P;
    droite = (rang + 1) % P;
    val = rang;
    for (int etape = 0; etape < P; etape++) {
        MPI_Isend(&val, 1, MPI_INT, droite, 0, MPI_COMM_WORLD, &req1);
        MPI_Irecv(&tmp, 1, MPI_INT, gauche, 0, MPI_COMM_WORLD, &req2);
        MPI_Wait(&req1, MPI_STATUS_IGNORE);
        MPI_Wait(&req2, MPI_STATUS_IGNORE);
        val = tmp;
    }
    printf("rang %d termine avec la valeur %d\n", rang, val);
    MPI_Finalize();
    return 0;
}
```

Reponse : Ce programme initialise les voisins d'anneau, repete `P` echanges non bloquants, puis remplace la valeur locale a chaque etape. / 答：该程序先计算环中的邻居，再重复 `P` 次非阻塞交换，并在每一步更新本地值。

2.10 [Impl:Anneau non bloquant] Echange entre voisins. / 环形相邻进程非阻塞交换。

```
#include <mpi.h>
#include <stdio.h>
int main(int argc, char **argv) {
    int rang, P, send_val, recv_val = -1, gauche, droite; MPI_Request req1, req2;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rang);
    MPI_Comm_size(MPI_COMM_WORLD, &P);
    gauche = (rang + P - 1) % P;
    droite = (rang + 1) % P;
    send_val = rang;
    MPI_Isend(&send_val, 1, MPI_INT, droite, 0, MPI_COMM_WORLD, &req1);
    MPI_Irecv(&recv_val, 1, MPI_INT, gauche, 0, MPI_COMM_WORLD, &req2);
    MPI_Wait(&req1, MPI_STATUS_IGNORE);
    MPI_Wait(&req2, MPI_STATUS_IGNORE);
    printf("rang %d a envoye %d a %d et reçu %d de %d\n", rang, send_val, droite, recv_val, gauche);
    MPI_Finalize();
    return 0;
}
```

Non-Bloquant Et Messages De Taille Inconnue

2.11 [Question:Isend buffer] Pourquoi ne doit-on pas reecrir le buffer juste apres MPI_Isend ? / 为什么 MPI_Isend 后不能立刻改写发送缓冲区？

Reponse : Parce que MPI_Isend ne garantit pas que la communication soit deja terminee au retour de l'appel. Tant que la requete n'est pas terminee, MPI peut encore avoir besoin des donnees du tampon d'envoi. / 答：因为 MPI_Isend 返回时并不保证通信已结束；在请求完成前，MPI 仍可能需要读取发送缓冲区中的内容。

Exemple complet : voir 2.10 .

2.12 [Question:Irecv buffer] Pourquoi ne doit-on pas lire le buffer juste apres MPI_Irecv ? / 为什么 MPI_Irecv 后不能立刻读取接收缓冲区？

Reponse : Parce que MPI_Irecv ne garantit pas que les donnees aient deja ete placees dans le tampon de reception. Il faut attendre la fin de la requete avant de lire ce tampon. / 答：因为 MPI_Irecv 返回时并不保证数据已经写入接收缓冲区，必须等请求完成后才能读取。

Exemple complet : voir 2.21 .

2.13 [Impl:ANY_SOURCE] Montrer comment recevoir avec MPI_ANY_SOURCE . / 说明如何使用 MPI_ANY_SOURCE 。

```
#include <mpi.h>
#include <stdio.h>
int main(int argc, char **argv) {
    int rang, x = -1; MPI_Status sta;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rang);
    if (rang == 0) {
        MPI_Send(&rang, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
    } else if (rang == 1) {
        MPI_Recv(&x, 1, MPI_INT, MPI_ANY_SOURCE, 0, MPI_COMM_WORLD, &sta);
        printf("rang 1 a reçu %d depuis %d\n", x, sta.MPI_SOURCE);
    }
    MPI_Finalize();
    return 0;
}
```

Reponse : La vraie source est ensuite disponible dans sta.MPI_SOURCE . / 答：实际发送者随后可通过 sta.MPI_SOURCE 得到。

2.14 [Impl:ANY_TAG] Montrer comment recevoir avec MPI_ANY_TAG . / 说明如何使用 MPI_ANY_TAG 。

```
#include <mpi.h>
#include <stdio.h>
int main(int argc, char **argv) {
    int rang, x = -1; MPI_Status sta;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rang);
    if (rang == 0) {
        int tag = 7;
        MPI_Send(&rang, 1, MPI_INT, 1, tag, MPI_COMM_WORLD);
    } else if (rang == 1) {
        MPI_Recv(&x, 1, MPI_INT, 0, MPI_ANY_TAG, MPI_COMM_WORLD, &sta);
        printf("rang 1 a reçu %d avec le tag %d\n", x, sta.MPI_TAG);
    }
    MPI_Finalize();
    return 0;
}
```

Reponse : Le vrai tag est ensuite disponible dans sta.MPI_TAG . / 答：实际标签随后可通过 sta.MPI_TAG 得到。

2.15 [Impl:Probe] Strategie pour une taille inconnue. / 未知长度消息的标准策略。

Reponse :

```
MPI_Status status;
int count;
int *buf;
MPI_Probe(0, 0, MPI_COMM_WORLD, &status);
MPI_Get_count(&status, MPI_INT, &count);
buf = (int *)malloc(count * sizeof(int));
MPI_Recv(buf, count, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
```

2.16 [Comparaison:Non bloquant vs asynchrone] Pourquoi "non bloquant" ne veut-il pas automatiquement dire "asynchrone" ? / 为什么“非阻塞”并不自动等于“异步”？

Reponse : "Non bloquant" decrit le fait que l'appel retourne avant la fin de la communication. Cela ne garantit pas a lui seul un recouvrement effectif entre calcul et communication ni une progression totalement en arriere-plan. / 答：“非阻塞”只说明调用会在通信完成前返回，它并不自动保证真正的后台推进或明显的通信/计算重叠。

Exemple complet : voir 2.10 .

Send / Ssend / Bsend

2.17 [Comparaison:Modes d'envoi] Comparer MPI_Send , MPI_Ssend et MPI_Bsend . / 比较 MPI_Send 、 MPI_Ssend 和 MPI_Bsend 。

Reponse : MPI_Send est un envoi standard : l'implementation choisit le protocole. MPI_Ssend est un envoi synchrone : il exige la mise en correspondance avec la reception. MPI_Bsend est un envoi bufferise : les donnees sont d'abord copiees dans un buffer attache par l'utilisateur. / 答：MPI_Send 是标准发送，具体协议由 MPI 实现决定；MPI_Ssend 是同步发送，要求与接收建立匹配；MPI_Bsend 是缓冲发送，数据先拷贝到用户附加的 MPI 缓冲区中。

Exemple complet : voir 2.18 .

2.18 [Debug:Petits vs grands messages] Exemple qui marche parfois mais peut bloquer. / 小消息能跑、大消息可能死锁的例子。

```
int voisin = (rang == 0) ? 1 : 0;
MPI_Send(buf1, N, MPI_BYTE, voisin, 0, MPI_COMM_WORLD);
MPI_Recv(buf2, N, MPI_BYTE, voisin, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
```

Reponse : Avec un petit message, `MPI_Send` peut etre bufferise. Avec un grand message, il peut devenir synchrone et le schema symetrique peut bloquer. / 答：小消息时 `MPI_Send` 可能走缓冲协议，大消息时可能转为同步协议，从而在这种对称写法下死锁。

2.19 [Question:Ssend] Quand utiliser `MPI_Ssend` plutot que `MPI_Send` ? / 什么时候更适合用 `MPI_Ssend` 而不是 `MPI_Send` ?

Reponse : On prefere `MPI_Ssend` quand on veut une semantique synchrone explicite, c'est-a-dire quand on veut que l'envoi ne se termine qu'apres la mise en correspondance avec la reception. Il est aussi utile pour reveler des interblocages latents pendant le debug. / 答：当希望显式采用同步发送语义时，更适合用 `MPI_Ssend`；它也常用于调试，以暴露潜在死锁。

Exemple complet : voir 2.18 .

2.20 [Question:Bsend] Quand utiliser `MPI_Bsend` ? / 什么时候适合用 `MPI_Bsend` ?

Reponse : `MPI_Bsend` est utile quand on veut decoupler davantage l'emetteur et le recepteur et autoriser le retour de l'envoi apres copie dans un buffer utilisateur attache a MPI. / 答：当希望发送方与接收方更解耦，并允许发送在数据拷入用户附加缓冲区后返回时，适合使用 `MPI_Bsend`。

Exemple complet : voir 2.18 .

Wait / Test / Requetes Multiples

2.21 [Impl:Waitall] Plusieurs `MPI_Irecv` puis `MPI_Waitall` . / 多个 `MPI_Irecv` 后统一 `MPI_Waitall` 。

```
#include <mpi.h>
#include <stdio.h>
int main(int argc, char **argv) {
    int rang, P, buf[3]; MPI_Request req[3]; MPI_Status sta[3];
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rang);
    MPI_Comm_size(MPI_COMM_WORLD, &P);
    if (P >= 4) {
        if (rang == 0) {
            MPI_Irecv(&buf[0], 1, MPI_INT, 1, 0, MPI_COMM_WORLD, &req[0]);
            MPI_Irecv(&buf[1], 1, MPI_INT, 2, 0, MPI_COMM_WORLD, &req[1]);
            MPI_Irecv(&buf[2], 1, MPI_INT, 3, 0, MPI_COMM_WORLD, &req[2]);
            MPI_Waitall(3, req, sta);
            printf("rang 0 a recu %d %d %d\n", buf[0], buf[1], buf[2]);
        } else if (rang >= 1 && rang <= 3) {
            MPI_Send(&rang, 1, MPI_INT, 0, 0, MPI_COMM_WORLD);
        }
    }
    MPI_Finalize();
    return 0;
}
```

2.22 [Strategie:Echange pair/impair] Strategie pour voisins pairs et impairs. / 奇偶相邻进程交换策略。

Reponse : On peut casser la symetrie ainsi : les rangs pairs envoient d'abord a `rang + 1` puis recoivent, tandis que les rangs impairs recoivent d'abord de `rang - 1` puis envoient. / 答：可以通过打破对称性来实现：偶数 rank 先向 `rang + 1` 发送再接收，奇数 rank 先从 `rang - 1` 接收再发送。

2.23 [Impl:Exemple minimal] Un reel de 0 vers 1. / 一个实数从 0 发到 1。

```
#include <mpi.h>
#include <stdio.h>
int main(int argc, char **argv) {
    int rang;
    double x = 3.14;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rang);
    if (rang == 0) {
        MPI_Send(&x, 1, MPI_DOUBLE, 1, 0, MPI_COMM_WORLD);
    } else if (rang == 1) {
        MPI_Recv(&x, 1, MPI_DOUBLE, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        printf("rang 1 a recu %.2f\n", x);
    }
    MPI_Finalize();
    return 0;
}
```

2.24 [Impl:Test] Polling avec `MPI_Test` . / 用 `MPI_Test` 轮询完成。

```
#include <mpi.h>
#include <stdio.h>
int main(int argc, char **argv) {
    int rang, x = -1, flag = 0; MPI_Request req;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rang);
    if (rang == 0) {
        x = 42;
        MPI_Send(&x, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
    } else if (rang == 1) {
        MPI_Irecv(&x, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, &req);
        while (!flag) {
            MPI_Test(&req, &flag, MPI_STATUS_IGNORE);
        }
        printf("rang 1 a recu %d\n", x);
    }
    MPI_Finalize();
    return 0;
}
```

2.25 [Comparaison:Waitall vs Waitany] Quelle difference d'usage entre `MPI_Waitall` et `MPI_Waitany` ? / `MPI_Waitall` 和 `MPI_Waitany` 的使用区别是什么？

Reponse : `MPI_Waitall` attend que toutes les requetes soient terminees. `MPI_Waitany` retourne des qu'une requete parmi l'ensemble est terminee et indique laquelle. / 答： `MPI_Waitall` 要等全部请求完成才返回； `MPI_Waitany` 只要其中一个请求完成就返回，并指出是哪一个。

Exemple complet : voir 2.21 .

Schemas Standards Et Variantes

2.26 [Impl:Iprobe -> Recv] Ecrire le schema standard non bloquant pour taille inconnue. / 写出未知长度消息的非阻塞探测流程。

```
int flag = 0, n;
int *buf;
MPI_Status sta;
while (!flag) {
    MPI_Iprobe(MPI_ANY_SOURCE, MPI_ANY_TAG, MPI_COMM_WORLD, &flag, &sta);
}
MPI_Get_count(&sta, MPI_INT, &n);
buf = (int *)malloc(n * sizeof(int));
MPI_Recv(buf, n, MPI_INT, sta.MPI_SOURCE, sta.MPI_TAG, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
```

2.27 [Impl:Probe -> Recv] Ecrire le schema standard bloquant pour taille inconnue. / 写出未知长度消息的阻塞探测流程。

```
int n;
int *buf;
MPI_Status sta;
MPI_Probe(MPI_ANY_SOURCE, MPI_ANY_TAG, MPI_COMM_WORLD, &sta);
MPI_Get_count(&sta, MPI_INT, &n);
buf = (int *)malloc(n * sizeof(int));
MPI_Recv(buf, n, MPI_INT, sta.MPI_SOURCE, sta.MPI_TAG, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
```

2.28 [Definition:Buffer attach] A quoi servent MPI_Buffer_attach et MPI_Buffer_detach ? / MPI_Buffer_attach 和 MPI_Buffer_detach 有什么作用？

Reponse : MPI_Buffer_attach sert a attacher a MPI un buffer utilisateur utilise par MPI_Bsend . MPI_Buffer_detach sert a detacher ce buffer et a recuperer son adresse et sa taille. / 答：MPI_Buffer_attach 用于给 MPI_Bsend 附加用户缓冲区；MPI_Buffer_detach 用于把这个缓冲区从 MPI 中分离，并取回其地址和大小。

2.29 [Question:Bsend buffer] Pourquoi ne faut-il pas detacher trop tot le buffer de MPI_Bsend ? / 为什么不能过早分离 MPI_Bsend 的缓冲区？

Reponse : Parce que des messages MPI_Bsend peuvent encore etre en cours et utiliser ce buffer. Le detacher ou le liberer trop tot rendrait le comportement incorrect. / 答：因为尚未完成的 MPI_Bsend 可能仍在使用该缓冲区；过早分离或释放会导致错误行为。

Exemple complet : voir 2.18 .

2.30 [Impl:Collecte sur 0] Envoyer tous les rangs vers le processus 0 avec Send/Recv . / 只用 Send/Recv 把所有 rank 发到 0。

```
#include <mpi.h>
#include <stdio.h>
int main(int argc, char **argv) {
    int rang, P;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rang);
    MPI_Comm_size(MPI_COMM_WORLD, &P);
    if (rang == 0) {
        int x;
        for (int src = 1; src < P; src++) {
            MPI_Recv(&x, 1, MPI_INT, src, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
            printf("rang 0 a recu %d depuis %d\n", x, src);
        }
    } else {
        MPI_Send(&rang, 1, MPI_INT, 0, 0, MPI_COMM_WORLD);
    }
    MPI_Finalize();
    return 0;
}
```

2.31 [Comparaison:Waitsome/Testany] Quand MPI_Waitsome ou MPI_Testany sont-ils plus naturels que MPI_Waitall ? / 什么时候 MPI_Waitsome 或 MPI_Testany 比 MPI_Waitall 更自然？

Reponse : Ils sont plus naturels quand on veut traiter les messages au fur et a mesure de leur arrivee, sans attendre la terminaison de toutes les communications en meme temps. / 答：当希望消息一到就逐个处理，而不是等所有通信全部结束后再统一处理时，MPI_Waitsome 或 MPI_Testany 更自然。

Exemple complet : voir 2.21 .

3. Analyse de Code et Correction / 3. 代码分析与改错

Validite Et Appels Collectifs

3.01 [Analyse:Destination invalide] Identifier le probleme principal et proposer une correction minimale pour l'extrait suivant. / 指出下面代码的主要问题，并给出最小修改方案。

```
int x = rang;
if (rang == 0) {
    MPI_Send(&x, 1, MPI_INT, 2, 0, MPI_COMM_WORLD);
}else if (rang == 2) {
    MPI_Recv(&x, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
}
```

Reponse : Probleme principal : le code suppose que le processus de rang 2 existe. Si $P < 3$, la destination est invalide. Correction minimale : n'executer cet envoi que si $P > 2$. / 答：主要问题是代码默认 rank 2 一定存在；若 $P < 3$ ，目标进程非法。最小修改是只在 $P > 2$ 时执行发送。

```
int x = rang;
if (P > 2 && rang == 0) {
    MPI_Send(&x, 1, MPI_INT, 2, 0, MPI_COMM_WORLD);
}else if (P > 2 && rang == 2) {
    MPI_Recv(&x, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
}
```

3.02 [Analyse:Racine incoherente] Identifier le probleme principal et proposer une correction minimale pour l'extrait suivant. / 指出下面代码的主要问题，并给出最小修改方案。

```
double s_loc = 0.0, s_glob;
MPI_Reduce(&s_loc, &s_glob, 1, MPI_DOUBLE, MPI_SUM, rang, MPI_COMM_WORLD);
```

Reponse : Probleme principal : la racine de MPI_Reduce n'est pas la meme sur tous les processus, car chaque processus passe rang . Correction minimale : choisir une racine unique, par exemple 0. / 答：主要问题是 MPI_Reduce 的 root 在各进程上不一致，因为每个进程都传了自己的 rang 。最小修改是统一 root，例如取 0 。

```
double s_loc = 0.0, s_glob;
MPI_Reduce(&s_loc, &s_glob, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);
```

3.03 [Analyse:Securite du buffer lsend] Identifier le probleme principal et proposer une correction minimale pour l'extrait suivant. / 指出下面代码的主要问题，并给出最小修改方案。

```
MPI_Request req;
int x = 10;
MPI_Isend(&x, 1, MPI_INT, 1, 0, MPI_COMM_WORLD, &req);
x = 20;
MPI_Wait(&req, MPI_STATUS_IGNORE);
```

Reponse : Probleme principal : le tampon d'envoi est modifie avant la fin de la communication non bloquante. Correction minimale : attendre la fin du `MPI_Isend` avant de modifier `x` . / 答：主要问题是非阻塞发送完成前就改写了发送缓冲区。最小修改是在改写 `x` 之前先等待 `MPI_Isend` 完成。

```
MPI_Request req;
int x = 10;
MPI_Isend(&x, 1, MPI_INT, 1, 0, MPI_COMM_WORLD, &req);
MPI_Wait(&req, MPI_STATUS_IGNORE);
x = 20;
```

3.04 [Analyse:Collective partielle] Identifier le probleme principal et proposer une correction minimale pour l'extrait suivant. / 指出下面代码的主要问题，并给出最小修改方案。

```
if (rang == 0) {
    MPI_Bcast(&rang, 1, MPI_INT, 0, MPI_COMM_WORLD);
}
```

Reponse : Probleme principal : `MPI_Bcast` est une communication collective et doit etre appelee par tous les processus du communicateur. Correction minimale : sortir l'appel de la branche conditionnelle. / 答：主要问题是 `MPI_Bcast` 属于集合通信，必须由该 communicator 中所有进程调用。最小修改是把它移出条件分支。

```
MPI_Bcast(&rang, 1, MPI_INT, 0, MPI_COMM_WORLD);
```

Corrections Minimales

3.05 [Correction:Interblocage en anneau] Identifier le probleme principal et proposer une correction minimale pour l'extrait suivant. / 指出下面代码的主要问题，并给出最小修改方案。

```
MPI_Status sta;
int l, r, w;
w = rang;
l = (rang + P - 1) % P;
r = (rang + 1) % P;
MPI_Send(&w, 1, MPI_INT, l, 0, MPI_COMM_WORLD);
MPI_Recv(&w, 1, MPI_INT, r, 0, MPI_COMM_WORLD, &sta);
```

Reponse : Probleme principal : tous les processus font `Send` puis `Recv` , ce qui peut provoquer un interblocage dans l'anneau. Correction minimale : utiliser un envoi et une reception non bloquants, puis attendre. / 答：主要问题是所有进程都先 `Send` 再 `Recv` , 环形结构下可能死锁。最小修改是改成非阻塞收发再等待完成。

```
MPI_Request req1, req2;
MPI_Status sta;
int l, r, w;
w = rang;
l = (rang + P - 1) % P;
r = (rang + 1) % P;
MPI_Isend(&w, 1, MPI_INT, l, 0, MPI_COMM_WORLD, &req1);
MPI_Irecv(&w, 1, MPI_INT, r, 0, MPI_COMM_WORLD, &req2);
MPI_Wait(&req1, MPI_STATUS_IGNORE);
MPI_Wait(&req2, &sta);
```

3.06 [Correction:Racine incoherente] Identifier le probleme principal et proposer une correction minimale pour l'extrait suivant. / 指出下面代码的主要问题，并给出最小修改方案。

```
double som_glob, som_loc = 0.0;
for (i = 0; i < N; i++) {
    som_loc += x[i];
}
MPI_Reduce(&som_loc, &som_glob, 1, MPI_DOUBLE, MPI_SUM, rang, MPI_COMM_WORLD);
```

Reponse : Probleme principal : la racine est incorrecte car `rang` varie selon le processus. Correction minimale : choisir une racine unique, par exemple `0` . / 答：主要问题是 root 参数错误，因为 `rang` 在不同进程上不同。最小修改是统一选一个 root，例如 `0` 。

```
double som_glob, som_loc = 0.0;
for (i = 0; i < N; i++) {
    som_loc += x[i];
}
MPI_Reduce(&som_loc, &som_glob, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);
```

3.07 [Correction:Buffers Allreduce] Identifier le probleme principal et proposer une correction minimale pour l'extrait suivant. / 指出下面代码的主要问题，并给出最小修改方案。

```
int data = rang;
MPI_Allreduce(&data, &data, 1, MPI_INT, MPI_SUM, MPI_COMM_WORLD);
```

Reponse : Probleme principal : la meme variable est utilisee en envoi et en reception. Correction minimale : utiliser deux variables distinctes. / 答：主要问题是同一个变量同时作为发送缓冲区和接收缓冲区。最小修改是分成两个变量。

```
int send_data = rang, recv_data;
MPI_Allreduce(&send_data, &recv_data, 1, MPI_INT, MPI_SUM, MPI_COMM_WORLD);
```

3.08 [Correction:Allocation Gather] Identifier le probleme principal et proposer une correction minimale pour l'extrait suivant. / 指出下面代码的主要问题，并给出最小修改方案。

```
int rank, size, local_data = rang, gathered_data[10];
MPI_Gather(&local_data, 1, MPI_INT, gathered_data, 1, MPI_INT, 0, MPI_COMM_WORLD);
```

Reponse : Probleme principal : le tableau de reception a une taille fixe qui peut etre insuffisante si `size > 10` . Correction minimale : allouer `gathered_data` en fonction de `size` sur la racine. / 答：主要问题是接收数组大小写死为 10，若 `size > 10` 就不够。最小修改是在根进程按 `size` 动态分配。

```
int local_data = rang;
int *gathered_data = NULL;
if (rang == 0) {
    gathered_data = (int *)malloc(size * sizeof(int));
}
MPI_Gather(&local_data, 1, MPI_INT, gathered_data, 1, MPI_INT, 0, MPI_COMM_WORLD);
```

3.09 [Correction:Collective partielle] Identifier le probleme principal et proposer une correction minimale pour l'extrait suivant. / 指出下面代码的主要问题，并给出最小修改方案。

```
int x;
if (rang == 0) x = 42;
if (rang == 0) {
    MPI_Bcast(&x, 1, MPI_INT, 0, MPI_COMM_WORLD);
}
```

Reponse : Probleme principal : `MPI_Bcast` n'est appelee que par la racine, alors qu'elle doit etre appelee par tous les processus. Correction minimale : laisser seulement l'initialisation dans le `if`, puis appeler `MPI_Bcast` partout. / 答：主要问题是只有 root 调用了 `MPI_Bcast`，而它必须由所有进程调用。最小修改是只把初始化留在条件分支里，广播调用放到外面。

```
int x;
if (rang == 0) x = 42;
MPI_Bcast(&x, 1, MPI_INT, 0, MPI_COMM_WORLD);
```

3.10 [Correction:Securite du buffer lsend] Identifier le probleme principal et proposer une correction minimale pour l'extrait suivant. / 指出下面代码的主要问题，并给出最小修改方案。

```
MPI_Request req;
double x = 1.0;
MPI_Isend(&x, 1, MPI_DOUBLE, 1, 0, MPI_COMM_WORLD, &req);
x = 2.0;
MPI_Wait(&req, MPI_STATUS_IGNORE);
```

Reponse : Probleme principal : le tampon d'envoi est modifie avant la fin du `MPI_Isend`. Correction minimale : attendre avant de modifier `x`. / 答：主要问题是发送缓冲区在 `MPI_Isend` 完成前就被改写了。最小修改是先等待再修改 `x`。

```
MPI_Request req;
double x = 1.0;
MPI_Isend(&x, 1, MPI_DOUBLE, 1, 0, MPI_COMM_WORLD, &req);
MPI_Wait(&req, MPI_STATUS_IGNORE);
x = 2.0;
```

3.11 [Correction:Securite du buffer lrecv] Identifier le probleme principal et proposer une correction minimale pour l'extrait suivant. / 指出下面代码的主要问题，并给出最小修改方案。

```
MPI_Request req;
int x;
MPI_Irecv(&x, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, &req);
printf("%d\n", x);
MPI_Wait(&req, MPI_STATUS_IGNORE);
```

Reponse : Probleme principal : le tampon de reception est lu avant la fin de la communication non bloquante. Correction minimale : faire `MPI_wait` avant le `printf`. / 答：主要问题是在非阻塞接收完成前就读取了接收缓冲区。最小修改是先 `MPI_Wait` 再打印。

```
MPI_Request req;
int x;
MPI_Irecv(&x, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, &req);
MPI_Wait(&req, MPI_STATUS_IGNORE);
printf("%d\n", x);
```

Pieges Mixtes Et Cas Limites

3.12 [Analyse:Barrier mal placee] Identifier le probleme principal et proposer une correction minimale pour l'extrait suivant. / 指出下面代码的主要问题，并给出最小修改方案。

```
int x = rang;
if (rang == 0) {
    MPI_Send(&x, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
}
MPI_Barrier(MPI_COMM_WORLD);
if (rang == 1) {
    MPI_Recv(&x, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
}
```

Reponse : Probleme principal : la barriere est placee entre un envoi et la reception correspondante, ce qui peut bloquer le programme. Correction minimale : faire la reception avant la barriere, ou supprimer la barriere. / 答：主要问题是把屏障放在发送和对应接收之间，可能导致程序卡住。最小修改是把接收移到 barrier 前，或直接去掉 barrier。

```
int x = rang;
if (rang == 0) {
    MPI_Send(&x, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
}
if (rang == 1) {
    MPI_Recv(&x, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
}
MPI_Barrier(MPI_COMM_WORLD);
```

3.13 [Analyse:Hypothese sur P] Identifier le probleme principal et proposer une correction minimale pour l'extrait suivant. / 指出下面代码的主要问题，并给出最小修改方案。

```

MPI_Request req;
MPI_Status sta1, sta2;
double m = 1.0 / (rang + 1);
int v = 2;
if (rang == 0 || rang == 2) {
    if (rang == 2) v = 0;
    MPI_Isend(&m, 1, MPI_DOUBLE, v, 1000, MPI_COMM_WORLD, &req);
    MPI_Recv(&m, 1, MPI_DOUBLE, v, 1000, MPI_COMM_WORLD, &sta1);
    MPI_Wait(&req, &sta2);
}

```

Reponse : Probleme principal : le code suppose implicitement que le processus 2 existe. Correction minimale : proteger le schema par un test `P > 2` . / 答：主要问题是代码默认 rank 2 存在。最小修改是在 communicator 至少有 3 个进程时才执行这段通信。

```

MPI_Request req;
MPI_Status sta1, sta2;
double m = 1.0 / (rang + 1);
int v = 2;
if (P > 2 && (rang == 0 || rang == 2)) {
    if (rang == 2) v = 0;
    MPI_Isend(&m, 1, MPI_DOUBLE, v, 1000, MPI_COMM_WORLD, &req);
    MPI_Recv(&m, 1, MPI_DOUBLE, v, 1000, MPI_COMM_WORLD, &sta1);
    MPI_Wait(&req, &sta2);
}

```

3.14 [Analyse:Collective partielle] Identifier le probleme principal et proposer une correction minimale pour l'extrait suivant. / 指出下面代码的主要问题，并给出最小修改方案。

```

int x = rang;
if (rang % 2 == 0) {
    MPI_Bcast(&x, 1, MPI_INT, 0, MPI_COMM_WORLD);
}
MPI_Reduce(&x, &x, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);

```

Reponse : Probleme principal : `MPI_Bcast` n'est appelee que par les rangs pairs, alors qu'une collective doit etre appelee par tous les processus du communicateur. Correction minimale : faire appeler `MPI_Bcast` par tout le monde. / 答：主要问题是只有偶数进程调用了 `MPI_Bcast` ，而集合通信必须由 communicator 中所有进程调用。最小修改是让所有进程都调用它。

```

int x = rang;
MPI_Bcast(&x, 1, MPI_INT, 0, MPI_COMM_WORLD);
MPI_Reduce(&x, &x, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);

```

solution 2 : 保留偶数进程调用 `MPI_Bcast` 的想法，但把它放在一个新的 communicator 上，确保调用者一致。

```

MPI_Comm pair_comm;
int color = (rang % 2 == 0) ? 0 : MPI_UNDEFINED;
MPI_Comm_split(MPI_COMM_WORLD, color, rang, &pair_comm);

if (pair_comm != MPI_COMM_NULL) {
    int x = rang;
    MPI_Bcast(&x, 1, MPI_INT, 0, pair_comm);
    MPI_Comm_free(&pair_comm);
}
MPI_Reduce(&x, &x, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);

```

3.15 [Analyse:Partenaires incoherents] Identifier le probleme principal et proposer une correction minimale pour l'extrait suivant. / 指出下面代码的主要问题，并给出最小修改方案。

```

int x = rang;
MPI_Send(&x, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
MPI_Recv(&x, 1, MPI_INT, 1, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);

```

Reponse : Probleme principal : ce schema est faux pour presque tous les rangs, en particulier un processus peut s'envoyer a lui-meme ou viser un partenaire qui ne correspond pas a l'intention. Correction minimale : restreindre l'echange aux rangs 0 et 1 avec des partenaires coherents. / 答：主要问题是这个收发关系对大多数 rank 都不成立，甚至可能出现自发自收或和错误伙伴通信。最小修改是把交换限制在 rank 0 和 1，并明确对应关系。

```

int x = rang;
if (rang == 0) {
    MPI_Send(&x, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
    MPI_Recv(&x, 1, MPI_INT, 1, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
} else if (rang == 1) {
    MPI_Recv(&x, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    MPI_Send(&x, 1, MPI_INT, 0, 0, MPI_COMM_WORLD);
}

```

3.16 [Analyse:Racine mal initialisee] Identifier le probleme principal et proposer une correction minimale pour l'extrait suivant. / 指出下面代码的主要问题，并给出最小修改方案。

```

int x;
if (rang == 0) x = 1;
MPI_Bcast(&x, 1, MPI_INT, 1, MPI_COMM_WORLD);

```

Reponse : Probleme principal : la racine indiquee est 1 , mais la valeur n'est initialisee que sur le rang 0 . Correction minimale : soit initialiser sur le rang 1 , soit mettre `root = 0` . / 答：主要问题是广播 root 写成了 1，但只有 rank 0 初始化了数据。最小修改是把 root 改成 0，或改为在 rank 1 上初始化。

```

int x;
if (rang == 0) x = 1;
MPI_Bcast(&x, 1, MPI_INT, 0, MPI_COMM_WORLD);

```

3.17 [Analyse:Attente manquante] Identifier le probleme principal et proposer une correction minimale pour l'extrait suivant. / 指出下面代码的主要问题，并给出最小修改方案。

```
MPI_Request req1, req2;
int x = rang;
MPI_Isend(&x, 1, MPI_INT, (rang + 1) % P, 0, MPI_COMM_WORLD, &req1);
MPI_Irecv(&x, 1, MPI_INT, (rang + P - 1) % P, 0, MPI_COMM_WORLD, &req2);
printf("%d\n", x);
```

Reponse : Probleme principal : le programme affiche `x` avant la fin des communications non bloquantes. Correction minimale : attendre la terminaison avant d'utiliser `x`. / 答：主要问题是在非阻塞通信完成前就使用了变量 `x`。最小修改是先等待请求完成，再打印。

```
MPI_Request req1, req2;
int x = rang;
MPI_Isend(&x, 1, MPI_INT, (rang + 1) % P, 0, MPI_COMM_WORLD, &req1);
MPI_Irecv(&x, 1, MPI_INT, (rang + P - 1) % P, 0, MPI_COMM_WORLD, &req2);
MPI_Wait(&req1, MPI_STATUS_IGNORE);
MPI_Wait(&req2, MPI_STATUS_IGNORE);
printf("%d\n", x);
```

3.18 [Analyse:Racine incoherente] Identifier le probleme principal et proposer une correction minimale pour l'extrait suivant. / 指出下面代码的主要问题，并给出最小修改方案。

```
int *buf = NULL;
if (rang == 0) buf = malloc(P * sizeof(int));
MPI_Scatter(buf, 1, MPI_INT, &rang, 1, MPI_INT, 1, MPI_COMM_WORLD);
```

Reponse : Probleme principal : la racine de `MPI_Scatter` est `1`, mais le tampon d'envoi n'est alloue que sur le rang `0`. Correction minimale : utiliser `0` comme racine. / 答：主要问题是 `MPI_Scatter` 的 root 写成了 1，但只有 rank 0 分配了发送缓冲区。最小修改是把 root 改成 0。

```
int *buf = NULL;
if (rang == 0) buf = malloc(P * sizeof(int));
MPI_Scatter(buf, 1, MPI_INT, &rang, 1, MPI_INT, 0, MPI_COMM_WORLD);
```

3.19 [Analyse:Barrier partielle] Identifier le probleme principal et proposer une correction minimale pour l'extrait suivant. / 指出下面代码的主要问题，并给出最小修改方案。

```
if (rang == 0) {
    MPI_Barrier(MPI_COMM_WORLD);
}
```

Reponse : Probleme principal : `MPI_Barrier` est collective et ne peut pas etre appelee par un seul processus. Correction minimale : faire appeler la barriere par tous les processus. / 答：主要问题是 `MPI_Barrier` 属于集合通信，不能只由一个进程调用。最小修改是让所有进程都调用 `barrier`。

```
MPI_Barrier(MPI_COMM_WORLD);
```

3.20 [Analyse:Voisin hors bornes] Identifier le probleme principal et proposer une correction minimale pour l'extrait suivant. / 指出下面代码的主要问题，并给出最小修改方案。

```
int x = rang;
MPI_Send(&x, 1, MPI_INT, rang + 1, 0, MPI_COMM_WORLD);
```

Reponse : Probleme principal : pour le dernier processus, la destination `rang + 1` est hors bornes. Correction minimale : utiliser un voisin valide, par exemple `(rang + 1) % P`. / 答：主要问题是最后一个进程会向不存在的 `rang + 1` 发送消息。最小修改是改成合法邻居，例如 `(rang + 1) % P`。

```
int x = rang;
MPI_Send(&x, 1, MPI_INT, (rang + 1) % P, 0, MPI_COMM_WORLD);
```

3.21 [Analyse:Recvbuf NULL] Identifier le probleme principal et proposer une correction minimale pour l'extrait suivant. / 指出下面代码的主要问题，并给出最小修改方案。

```
int x = rang;
MPI_Reduce(&x, NULL, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);
```

Reponse : Probleme principal : sur la racine, `recvbuf` ne peut pas etre `NULL`. Correction minimale : fournir une vraie variable de reception sur le rang `0`. / 答：主要问题是 root 进程上的 `recvbuf` 不能是 `NULL`。最小修改是在 rank 0 上提供真正的接收变量。

```
int x = rang, somme;
MPI_Reduce(&x, &somme, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);
```

3.22 [Analyse:Racine variable] Identifier le probleme principal et proposer une correction minimale pour l'extrait suivant. / 指出下面代码的主要问题，并给出最小修改方案。

```
int root = rang % 2;
int x = rang;
MPI_Gather(&x, 1, MPI_INT, NULL, 1, MPI_INT, root, MPI_COMM_WORLD);
```

Reponse : Probleme principal : la racine n'est pas la meme sur tous les processus, car `root` depend de `rang`. Correction minimale : choisir une racine unique, par exemple `0`, et fournir un vrai tampon de reception sur cette racine. / 答：主要问题是 root 随 `rang` 改变，不同进程上传入的 root 不一致。最小修改是统一 root，例如 0，并在根进程上提供真实接收缓冲区。

```
int root = 0;
int x = rang;
int *buf = NULL;
if (rang == root) buf = (int *)malloc(P * sizeof(int));
MPI_Gather(&x, 1, MPI_INT, buf, 1, MPI_INT, root, MPI_COMM_WORLD);
```

4. Communications Collectives / 4. 集合通信

Definitions Des Collectives

4.01 [Definition:Bcast] A quoi sert `MPI_Bcast` ? / `MPI_Bcast` 有什么作用？

Reponse : `MPI_Bcast` sert a diffuser une meme donnee depuis un processus racine vers tous les processus du communicateur. / 答：`MPI_Bcast` 用于把根进程上的同一份数据广播给通信子中的所有进程。

4.02 [Definition:Scatter] A quoi sert `MPI_Scatter` ? / `MPI_Scatter` 有什么作用？

Reponse : `MPI_Scatter` sert a distribuer depuis la racine un bloc de donnees a chaque processus, avec une meme taille par destinataire dans la forme simple. /

答：`MPI_Scatter` 用于由根进程向每个进程分发一块数据；在基础形式下，每块大小相同。

4.03 [Definition:Gather] A quoi sert `MPI_Gather` ? / `MPI_Gather` 有什么作用？

Reponse : `MPI_Gather` sert a rassembler sur la racine un bloc envoye par chaque processus. / 答：`MPI_Gather` 用于把每个进程的一块局部数据收集到根进程上。

4.04 [Definition:Reduce] A quoi sert `MPI_Reduce` ? / `MPI_Reduce` 有什么作用？

Reponse : `MPI_Reduce` sert a combiner les valeurs locales de tous les processus avec une operation comme `MPI_SUM` , `MPI_MAX` ou `MPI_MIN` , et a placer le resultat sur la racine. / 答 : `MPI_Reduce` 用于用 `MPI_SUM` 、 `MPI_MAX` 、 `MPI_MIN` 等操作把所有进程的局部值归约, 并把结果放到根进程上。

4.05 [Definition:Allreduce] A quoi sert `MPI_Allreduce` ? / `MPI_Allreduce` 有什么作用?

Reponse : `MPI_Allreduce` effectue la meme reduction que `MPI_Reduce` , mais le resultat final est rendu a tous les processus. / 答 : `MPI_Allreduce` 和 `MPI_Reduce` 语义相同, 但最终结果会给所有进程。

4.06 [Definition:Allgather] A quoi sert `MPI_Allgather` ? / `MPI_Allgather` 有什么作用?

Reponse : `MPI_Allgather` sert a rassembler les blocs locaux de tous les processus et a recopier le resultat complet sur chacun d'eux. / 答 : `MPI_Allgather` 用于汇集所有进程的局部块, 并让每个进程都拿到完整结果。

4.07 [Definition:Barrier] A quoi sert `MPI_Barrier` ? / `MPI_Barrier` 有什么作用?

Reponse : `MPI_Barrier` sert a synchroniser tous les processus d'un communicateur. Quand un processus en sort, tous les autres y sont deja entres. /

答 : `MPI_Barrier` 用于同步一个 communicator 中的所有进程; 某进程从 barrier 返回时, 可以确定所有进程都已经进入了该 barrier。

Exemple complet : voir 4.47 .

Exemples Simples

4.08 [Impl:Bcast] Ecrire un exemple simple de `MPI_Bcast` . / 写一个简单的 `MPI_Bcast` 例子。

```
int x;
if (rang == 0) x = 42;
MPI_Bcast(&x, 1, MPI_INT, 0, MPI_COMM_WORLD);
```

4.09 [Impl:Scatter] Ecrire un exemple simple de `MPI_Scatter` . / 写一个简单的 `MPI_Scatter` 例子。

```
int *tab = NULL, x;
if (rang == 0) tab = (int *)malloc(P * sizeof(int));
MPI_Scatter(tab, 1, MPI_INT, &x, 1, MPI_INT, 0, MPI_COMM_WORLD);
```

4.10 [Impl:Gather] Ecrire un exemple simple de `MPI_Gather` . / 写一个简单的 `MPI_Gather` 例子。

```
int x = rang;
int *tab = NULL;
if (rang == 0) tab = (int *)malloc(P * sizeof(int));
MPI_Gather(&x, 1, MPI_INT, tab, 1, MPI_INT, 0, MPI_COMM_WORLD);
```

4.11 [Impl:Reduce somme] Ecrire un exemple simple de `MPI_Reduce` avec `MPI_SUM` . / 写一个 `MPI_Reduce` + `MPI_SUM` 的简单例子。

```
int x = rang, somme;
MPI_Reduce(&x, &somme, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);
```

Comparaisons Et Raisonnements

4.12 [Comparaison:Reduce vs Allreduce] Expliquer la difference entre `MPI_Reduce` et `MPI_Allreduce` . / 解释 `MPI_Reduce` 与 `MPI_Allreduce` 的区别。

Reponse : `MPI_Reduce` laisse le resultat final seulement sur la racine. `MPI_Allreduce` donne ce resultat a tous les processus. / 答 : `MPI_Reduce` 只把结果放在根进程上; `MPI_Allreduce` 则把同样的结果给所有进程。

4.13 [Analyse:Participation collective] Expliquer pourquoi tous les processus doivent appeler la meme collective. / 解释为什么所有进程都必须调用同一个集合通信。

Reponse : Parce qu'une collective est definie au niveau du communicateur entier. Si certains processus n'appellent pas la meme operation, la synchronisation et le couplage des echanges deviennent incohérents et le programme peut bloquer. / 答 : 因为集合通信是在整个 communicator 层面定义的; 如果只有部分进程调用或调用的 collective 不一致, 通信匹配就会失去一致性并导致阻塞。

4.14 [Analyse:Collective conditionnelle] Expliquer pourquoi mettre une collective dans une branche conditionnelle dangereuse peut bloquer le programme. / 解释为什么把集合通信放在危险的条件分支里会导致阻塞。

Reponse : Si la condition n'est pas vraie sur tous les processus, certains appelleront la collective et d'autres non. Comme une collective exige une participation coherente, cela peut provoquer un blocage. / 答 : 若条件在不同进程上结果不同, 就会出现只有部分进程调用 collective 的情况; 而 collective 要求一致参与, 因此会死锁。

Schemas Algorithmiques

4.15 [Algorithme:Bcast -> Scatter -> Reduce] Concevoir une suite `Bcast -> Scatter -> calcul local -> Reduce` . / 设计一套 `Bcast -> Scatter -> 本地计算 -> Reduce` 流程。

Reponse : La racine initialise d'abord les parametres globaux puis les diffuse avec `MPI_Bcast` . Ensuite elle distribue les donnees par blocs avec `MPI_Scatter` . Chaque processus effectue son calcul local, puis `MPI_Reduce` combine les resultats locaux sur la racine. / 答 : 先由根进程初始化全局参数并用 `MPI_Bcast` 广播, 再用 `MPI_Scatter` 按块分发数据; 各进程本地计算后, 用 `MPI_Reduce` 在根进程上合并结果。

4.19 [Impl:Bcast -> Scatter -> Reduce] Ecrire un squelette MPI correspondant a cette suite d'operations. / 为这套 `Bcast -> Scatter -> calcul local -> Reduce` 写一个 MPI 实现框架。

```
/* Diffuser les parametres, distribuer les donnees, puis reduire le resultat. */
int N, local_n;
int *tab = NULL, *bloc;
int local_sum = 0, global_sum = 0;
if (rang == 0) N = 100;
MPI_Bcast(&N, 1, MPI_INT, 0, MPI_COMM_WORLD);
local_n = N / P;
bloc = (int *)malloc(local_n * sizeof(int));
if (rang == 0) tab = (int *)malloc(N * sizeof(int));
MPI_Scatter(tab, local_n, MPI_INT, bloc, local_n, MPI_INT, 0, MPI_COMM_WORLD);
for (int i = 0; i < local_n; i++) local_sum += bloc[i];
MPI_Reduce(&local_sum, &global_sum, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);
```

4.16 [Algorithme:Bcast -> Allreduce] Concevoir une suite `Bcast -> calcul local -> Allreduce` . / 设计一套 `Bcast -> 本地计算 -> Allreduce` 流程。

Reponse : On diffuse d'abord les informations communes avec `MPI_Bcast` , puis chaque processus effectue un calcul local, et enfin `MPI_Allreduce` produit un resultat global disponible sur tous les processus. / 答 : 先用 `MPI_Bcast` 广播公共信息, 再做本地计算, 最后用 `MPI_Allreduce` 得到所有进程都可见的全局结果。

4.20 [Impl:Bcast -> Allreduce] Ecrire un squelette MPI correspondant a cette suite d'operations. / 为这套 `Bcast -> calcul local -> Allreduce` 写一个 MPI 实现框架。

```
/* Diffuser les parametres, calculer localement, puis partager le resultat global. */
int N;
double local_val = 0.0, global_val = 0.0;
if (rang == 0) N = 100;
MPI_Bcast(&N, 1, MPI_INT, 0, MPI_COMM_WORLD);
local_val = rang + N;
MPI_Allreduce(&local_val, &global_val, 1, MPI_DOUBLE, MPI_SUM, MPI_COMM_WORLD);
```

4.17 [Algorithme:Scatter -> Gather] Concevoir une suite `Scatter -> calcul local -> Gather` . / 设计一套 `Scatter -> 本地计算 -> Gather` 流程。

Reponse : La racine distribue les donnees avec `MPI_Scatter` . Chaque processus calcule un resultat local a partir de son bloc, puis `MPI_Gather` rassemble ces resultats locaux sur la racine. / 答 : 根进程先用 `MPI_Scatter` 分发数据, 各进程基于本地块做计算, 然后用 `MPI_Gather` 把局部结果收回根进程。

4.21 [Impl:Scatter -> Gather] Ecrire un squelette MPI correspondant a cette suite d'operations. / 为这套 Scatter -> calcul local -> Gather 写一个 MPI 实现框架。

```
/* Distribuer les blocs, calculer localement, puis rassembler sur la racine. */
int N = 100, local_n = N / P;
int *tab = NULL, *bloc = (int *)malloc(local_n * sizeof(int));
int local_res = 0, *res = NULL;
if (rang == 0) {
    tab = (int *)malloc(N * sizeof(int));
    res = (int *)malloc(P * sizeof(int));
}
MPI_Scatter(tab, local_n, MPI_INT, bloc, local_n, MPI_INT, 0, MPI_COMM_WORLD);
for (int i = 0; i < local_n; i++) local_res += bloc[i];
MPI_Gather(&local_res, 1, MPI_INT, res, 1, MPI_INT, 0, MPI_COMM_WORLD);
```

4.18 [Algorithm:Reduce -> Bcast] Concevoir une suite Reduce -> Bcast et expliquer quand elle peut etre remplacee par Allreduce . / 设计 Reduce -> Bcast , 并解释何时可用 Allreduce 取代。

Reponse : Chaque processus calcule une valeur locale, MPI_Reduce donne le resultat global sur la racine, puis MPI_Bcast redistribue ce resultat a tous. Cette suite peut etre remplacee par MPI_Allreduce quand tous les processus doivent connaitre le resultat final. / 答：各进程先算局部值，再用 MPI_Reduce 在 root 上得到全局结果，然后用 MPI_Bcast 把它发给所有进程；若所有进程都需要最终结果，可直接改为 MPI_Allreduce。

4.22 [Impl:Reduce -> Bcast] Ecrire un squelette MPI correspondant a cette suite d'operations. / 为这套 Reduce -> Bcast 写一个 MPI 实现框架。

```
/* Calculer localement, reduire sur la racine, puis rediffuser le resultat. */
int local_val = rang, global_val = 0;
MPI_Reduce(&local_val, &global_val, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);
MPI_Bcast(&global_val, 1, MPI_INT, 0, MPI_COMM_WORLD);
```

4.23 [Comparaison:Send/Recv vs collective] Comparer une reconstruction manuelle en Send/Recv avec une primitive collective MPI. / 比较手工 Send/Recv 实现和 MPI 集合原语。

Reponse : Une reconstruction manuelle avec Send/Recv est possible, mais elle est plus verbeuse, plus delicate a raisonner et plus facile a rendre incorrecte. Les collectives MPI expriment directement le motif de communication et sont en general mieux optimisees. / 答：手工用 Send/Recv 重建集合通信是可行的，但代码更长、更容易出错、也更难分析；MPI 提供的 collective 更直接，也通常有更好的实现优化。

Programmation Courante

4.24 [Impl:Allreduce] Ecrire un exemple simple de MPI_Allreduce . / 写一个简单的 MPI_Allreduce 示例。

```
int x = rang, somme;
MPI_Allreduce(&x, &somme, 1, MPI_INT, MPI_SUM, MPI_COMM_WORLD);
```

4.25 [Impl:Diffusion de N] Montrer comment diffuser N initialise sur le rang 0 a tous les processus. / 说明如何把在 rank 0 初始化的 N 广播给所有进程。

```
int N;
if (rang == 0) N = 100;
MPI_Bcast(&N, 1, MPI_INT, 0, MPI_COMM_WORLD);
```

4.26 [Impl:Distribution en blocs] Montrer comment distribuer un tableau de taille N en blocs egaux avec MPI_Scatter . / 说明如何用 MPI_Scatter 平均分大小为 N 的数组。

```
int local_n = N / P;
int *tab = NULL;
int *bloc = (int *)malloc(local_n * sizeof(int));
if (rang == 0) tab = (int *)malloc(N * sizeof(int));
MPI_Scatter(tab, local_n, MPI_INT, bloc, local_n, MPI_INT, 0, MPI_COMM_WORLD);
```

4.27 [Impl:Rassemblement sur racine] Montrer comment rassembler des resultats locaux sur le rang 0 avec MPI_Gather . / 说明如何用 MPI_Gather 把局部结果收集到 rank 0。

```
int local_res = rang;
int *res = NULL;
if (rang == 0) res = (int *)malloc(P * sizeof(int));
MPI_Gather(&local_res, 1, MPI_INT, res, 1, MPI_INT, 0, MPI_COMM_WORLD);
```

4.28 [Impl:Somme globale] Montrer comment calculer une somme globale avec MPI_Reduce . / 说明如何用 MPI_Reduce 计算全局和。

```
int local_sum = rang, global_sum;
MPI_Reduce(&local_sum, &global_sum, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);
```

4.29 [Impl:Maximum global] Montrer comment calculer un maximum global avec MPI_Reduce et MPI_MAX . / 说明如何用 MPI_Reduce 和 MPI_MAX 计算全局最大值。

```
int local_max = rang, global_max;
MPI_Reduce(&local_max, &global_max, 1, MPI_INT, MPI_MAX, 0, MPI_COMM_WORLD);
```

4.30 [Impl:Minimum global] Montrer comment calculer un minimum global avec MPI_Reduce et MPI_MIN . / 说明如何用 MPI_Reduce 和 MPI_MIN 计算全局最小值。

```
int local_min = rang, global_min;
MPI_Reduce(&local_min, &global_min, 1, MPI_INT, MPI_MIN, 0, MPI_COMM_WORLD);
```

Faux Usages Et Coûts

4.31 [Analyse:Bcast conditionnel] Donner un exemple faux d'utilisation de MPI_Bcast dans une branche conditionnelle. / 给出一个把 MPI_Bcast 放进条件分支导致错误的例子。

```
int x;
if (rang == 0) {
    x = 1;
    MPI_Bcast(&x, 1, MPI_INT, 0, MPI_COMM_WORLD);
}
```

C'est faux car seuls certains processus appellent la collective. / 这是错的，因为只有部分进程调用了 collective。

4.32 [Analyse:Racines differentes] Donner un exemple faux d'utilisation de MPI_Reduce avec des racines differentes. / 给出一个使用不同 root 调用 MPI_Reduce 的错误例子。

```
int x = rang, somme;
MPI_Reduce(&x, &somme, 1, MPI_INT, MPI_SUM, rang, MPI_COMM_WORLD);
```

C'est faux car la racine doit etre identique sur tous les processus. / 这是错的，因为 root 必须在所有进程上保持一致。

4.33 [Comparaison:Allgather vs Gather] Expliquer quand choisir MPI_Allgather plutot que MPI_Gather . / 说明什么时候应选择 MPI_Allgather 而不是

MPI_Gather 。

Reponse : On choisit **MPI_Allgather** quand tous les processus ont besoin du tableau global reconstitue, et non pas seulement la racine. / 答：当所有进程都需要完整汇总结果，而不只是根进程需要时，应选择 **MPI_Allgather** 。

4.34 [Comparaison:Principe Allgather] Expliquer le principe de MPI_Allgather . / 解释 MPI_Allgather 的原理。

Reponse : Chaque processus fournit un bloc local, et l'operation reconstruit la concatenation de tous ces blocs dans un tampon de reception present sur chaque processus. / 答：每个进程提供一个局部块，**MPI_Allgather** 会把所有局部块拼接成完整结果，并在每个进程上都得到这份结果。

4.35 [Impl:Barrier entre phases] Ecrire un programme qui utilise MPI_Barrier entre deux phases de calcul fictives. / 写一个程序，在两段虚拟计算之间使用 MPI_Barrier 。

```
calcul_phase_1();
MPI_Barrier(MPI_COMM_WORLD);
calcul_phase_2();
```

Primitives Variees Et Variantes

4.36 [Definition:Alltoall] A quoi sert MPI_Alltoall ? / MPI_Alltoall 有什么作用？

Reponse : **MPI_Alltoall** sert a faire echanger a chaque processus un bloc avec tous les autres processus. Chaque processus envoie un bloc a chacun et recoit un bloc de chacun. / 答：**MPI_Alltoall** 用于每个进程和所有其他进程互相交换数据块；每个进程给每个目标发一块，也从每个源收一块。

Exemple complet : voir 4.46 .

4.37 [Comparaison:Scatter vs Scatterv] Expliquer la difference entre MPI_Scatter et MPI_Scatterv . / 解释 MPI_Scatter 与 MPI_Scatterv 的区别。

Reponse : **MPI_Scatter** suppose une meme taille de bloc pour tous les processus. **MPI_Scatterv** permet des tailles variables et utilise des tableaux **sendcounts** et **displs** . / 答：**MPI_Scatter** 要求发给每个进程的块大小一致；**MPI_Scatterv** 支持不同大小，并通过 **sendcounts** 和 **displs** 指定。

Exemple complet : voir 4.48 .

4.38 [Comparaison:Gather vs Gatherv] Expliquer la difference entre MPI_Gather et MPI_Gatherv . / 解释 MPI_Gather 与 MPI_Gatherv 的区别。

Reponse : **MPI_Gather** collecte des blocs de meme taille. **MPI_Gatherv** permet de collecter des blocs de tailles variables, avec **recvcounts** et **displs** . /

答：**MPI_Gather** 收集等长块；**MPI_Gatherv** 支持不同长度的块，并由 **recvcounts** 和 **displs** 描述布局。

Exemple complet : voir 4.49 .

4.39 [Comparaison:Alltoall vs Alltoallv] Expliquer la difference entre MPI_Alltoall et MPI_Alltoallv . / 解释 MPI_Alltoall 与 MPI_Alltoallv 的区别。

Reponse : **MPI_Alltoall** suppose un meme nombre d'elements echanges avec chaque partenaire. **MPI_Alltoallv** autorise des tailles variables selon le partenaire. / 答：**MPI_Alltoall** 假定和每个伙伴交换的数据量相同；**MPI_Alltoallv** 则允许按伙伴不同而变化。

Exemple complet : voir 4.46 .

4.40 [Analyse:Cout memoire Allgather] Expliquer le cout memoire de MPI_Allgather . / 解释 MPI_Allgather 的内存代价。

Reponse : Le resultat global complet est stocke sur chaque processus. Le cout memoire est donc plus eleve que pour **MPI_Gather** , ou seul le processus racine stocke l'ensemble. / 答：**MPI_Allgather** 会让每个进程都保存完整结果，因此内存开销比只在 root 上保存结果的 **MPI_Gather** 更高。

4.41 [Analyse:Cout Alltoall] Expliquer pourquoi MPI_Alltoall est souvent plus couteux que MPI_Bcast ou MPI_Reduce . / 解释为什么 MPI_Alltoall 往往比 MPI_Bcast 或 MPI_Reduce 更昂贵。

Reponse : Parce que le motif d'echange est beaucoup plus dense : chaque processus communique avec tous les autres. Le volume global d'echange augmente donc fortement avec **p** . / 答：因为 **MPI_Alltoall** 的通信图更密集，每个进程都要和所有其他进程交换数据，所以总通信量会随着 **p** 快速增加。

Exemple complet : voir 4.46 .

Applications Avancees

4.42 [Impl:Alltoall] Ecrire un exemple simple de MPI_Alltoall ou chaque processus envoie son rang a tous les autres. / 写一个简单的 MPI_Alltoall 示例，使每个进程把自己的值发给所有其他进程。

```
int *sendbuf = (int *)malloc(P * sizeof(int));
int *recvbuf = (int *)malloc(P * sizeof(int));
for (int p = 0; p < P; p++) sendbuf[p] = rang;
MPI_Alltoall(sendbuf, 1, MPI_INT, recvbuf, 1, MPI_INT, MPI_COMM_WORLD);
```

4.43 [Algorithme:Transposee distribuee] Expliquer comment utiliser MPI_Alltoall pour realiser la transposee distribuee d'une matrice de blocs. / 解释如何利用 MPI_Alltoall 实现分布式块矩阵转置。

Reponse : Si chaque processus possede une ligne de blocs, **MPI_Alltoall** permet d'envoyer au processus **p** le bloc qui doit devenir un element de sa colonne. L'effet global est une transposition distribuee des blocs. / 答：若每个进程持有一行块，**MPI_Alltoall** 可以把应当属于进程 **p** 列位置的块发给它，从而实现块矩阵的分布式转置。

4.44 [Impl:Schema Scatterv] Donner le schema standard d'utilisation de MPI_Scatterv avec tableaux sendcounts et displs . / 给出 MPI_Scatterv 配合 sendcounts 和 displs 的标准使用框架。

```
int *sendbuf = NULL;
int *sendcounts = NULL;
int *displs = NULL;
int *recvbuf = (int *)malloc(local_n * sizeof(int));
MPI_Scatterv(sendbuf, sendcounts, displs, MPI_INT,
             recvbuf, local_n, MPI_INT, 0, MPI_COMM_WORLD);
```

4.45 [Impl:Schema Gatherv] Donner le schema standard d'utilisation de MPI_Gatherv avec tableaux recvcounts et displs . / 给出 MPI_Gatherv 配合 recvcounts 和 displs 的标准使用框架。

```
int *sendbuf = local_buf;
int *recvbuf = NULL;
int *recvcounts = NULL;
int *displs = NULL;
MPI_Gatherv(sendbuf, local_n, MPI_INT,
            recvbuf, recvcounts, displs, MPI_INT, 0, MPI_COMM_WORLD);
```

4.46 [Application:Repartition non divisible] Proposer une repartition correcte de N elements quand N n'est pas divisible par P , puis indiquer quand utiliser MPI_Scatterv . / 设计 N 不能被 P 整除时的正确分配方案，并说明何时应使用 MPI_Scatterv 。

Reponse : On peut donner $N / P + 1$ elements aux premiers processus et N / P aux autres. On utilise **MPI_Scatterv** quand les tailles locales ne sont plus toutes identiques. / 答：可以让前若干个进程拿到 $N / P + 1$ 个元素，其余进程拿 N / P 个；只要各进程块大小不再一致，就应使用 **MPI_Scatterv** 。

Operateurs Speciaux

4.47 [Definition:MAXLOC/MINLOC] Expliquer a quoi servent MPI_MAXLOC et MPI_MINLOC . / 解释 MPI_MAXLOC 和 MPI_MINLOC 的作用。

Reponse : **MPI_MAXLOC** et **MPI_MINLOC** servent a reduire simultanement une valeur et sa position associee. Ils permettent par exemple d'obtenir le maximum global et l'indice ou il apparait. / 答：**MPI_MAXLOC** 和 **MPI_MINLOC** 用于同时归约“值”和“对应位置”；例如可同时求出全局最大值及其位置。

4.48 [Application:MAXLOC] Concevoir un exercice ou l'on cherche la valeur maximale globale et le rang qui la detient. / 设计一道题：求全局最大值以及持有该值的进程 rank。

Reponse : Chaque processus calcule sa valeur locale, construit une paire (valeur, rang) , puis on applique une reduction avec **MPI_MAXLOC** pour obtenir sur la racine la valeur maximale globale et le rang qui la detient. / 答：每个进程先得到本地值，再构造 (值, rank) 这样的二元组，然后用 **MPI_MAXLOC** 归约，就能在根进程上得到全局最大值及其持有者 rank。

4.49 [Definition:Operateurs de reduction] Donner un exemple d'operateur predefini logique ou bit a bit utilisable avec MPI_Reduce . / 给出一个可与 MPI_Reduce 搭配的逻辑或按位预定义操作例子。

Reponse : On peut citer `MPI LAND` pour un ET logique, `MPI LOR` pour un OU logique, ou encore `MPI BAND` pour un ET bit a bit. / 答：例如 `MPI LAND`（逻辑与）、`MPI LOR`（逻辑或）或 `MPI BAND`（按位与）都可以和 `MPI Reduce` 搭配使用。
Exemple complet : voir `4.48` , `4.25` et `4.46` .

5. Communicateurs / 5. 通信器

Definitions Des Communicateurs

5.01 [Definition:Communicateur] Qu'est-ce qu'un communicateur ? / 什么是 communicator?

Reponse : Un communicateur est un objet MPI qui represente un groupe de processus pouvant communiquer entre eux de maniere isolee. Il sert a organiser les communications dans une application parallele. / 答：communicator 是 MPI 中表示“一组可以彼此通信的进程”的对象，用于组织并行程序中的通信。
Exemple complet : voir la question 2 de [APPPartielCorr2024.md](#).

5.02 [Definition:Nouveau communicateur] Pourquoi creer un nouveau communicateur ? / 为什么创建新的 communicator?

Reponse : On cree un nouveau communicateur pour regrouper certains processus, isoler un espace de communication et eviter d'ecrire beaucoup de conditions dans les communications collectives. / 答：创建新 communicator 是为了把部分进程分组、隔离通信空间，并避免在 collective 中写大量条件判断。
Exemple complet : voir la question 2 de [APPPartielCorr2024.md](#).

5.03 [Definition:Comm_split] A quoi sert MPI_Comm_split ? / MPI_Comm_split 有什么作用？

Reponse : `MPI_Comm_split` sert a separer un communicateur en plusieurs sous-communicateurs selon `color` , puis a fixer l'ordre des rangs locaux selon `key` . / 答： `MPI_Comm_split` 用于按 `color` 把一个 communicator 划分成多个子 communicator，再按 `key` 决定组内的新 rank 顺序。
Exemple complet : voir la question 2 de [APPPartielCorr2024.md](#).

5.04 [Definition:Color] Quel est le role de color ? / color 起什么作用？

Reponse : `color` decide a quel sous-groupe appartient un processus. Les processus qui ont la meme valeur de `color` se retrouvent dans le meme nouveau communicateur. / 答： `color` 用来决定进程被分到哪一组； `color` 相同的进程会进入同一个新 communicator。
Exemple complet : voir la question 2 de [APPPartielCorr2024.md](#).

5.05 [Definition:Key] Quel est le role de key ? / key 起什么作用？

Reponse : `key` decide l'ordre des rangs dans le nouveau communicateur. Plus `key` est petit, plus le nouveau rang local est place tot. / 答： `key` 用于决定新 communicator 内的 rank 排序； `key` 越小，新局部 rank 越靠前。
Exemple complet : voir la question 2 de [APPPartielCorr2024.md](#).

Programmation De Base

5.06 [Impl:Split pair/impair] Separer les processus pairs et impairs avec MPI_Comm_split . / 用 MPI_Comm_split 分离奇偶进程。

```
MPI_Comm new_comm;
MPI_Comm_split(MPI_COMM_WORLD, rang % 2, rang, &new_comm);
```

5.07 [Impl:Rang local] Obtenir le rang local dans un sous-communicateur. / 获取子 communicator 中的本地 rank。

```
int local_rank;
MPI_Comm_rank(new_comm, &local_rank);
```

5.08 [Impl:Taille locale] Obtenir la taille d'un sous-communicateur. / 获取子 communicator 的大小。

```
int local_size;
MPI_Comm_size(new_comm, &local_size);
```

Analyse Et Proprietes

5.09 [Analyse:Comm_split collectif] Expliquer pourquoi MPI_Comm_split est collectif. / 解释为什么 MPI_Comm_split 是集合操作。

Reponse : Parce que la construction du nouveau communicateur concerne tout le groupe de processus de l'ancien communicateur. MPI doit connaitre la participation de tous pour construire les sous-groupes de maniere coherente. / 答：因为新 communicator 的构造涉及原 communicator 中的整个进程组；MPI 需要知道所有参与进程的分组信息，才能一致地创建子组。
Exemple complet : voir la question 2 de [APPPartielCorr2024.md](#).

5.10 [Definition:MPI_UNDEFINED] Expliquer ce qui se passe avec MPI_UNDEFINED . / 解释 MPI_UNDEFINED 的效果。

Reponse : Si un processus passe `MPI_UNDEFINED` comme `color` , il n'entre dans aucun nouveau communicateur et recoit `MPI_COMM_NULL` . / 答：如果某个进程把 `color` 设为 `MPI_UNDEFINED` ，它就不会加入任何新 communicator，返回值会是 `MPI_COMM_NULL` 。
Exemple complet : voir la question 2 de [APPPartielCorr2024.md](#).

5.11 [Analyse:Rangs multiples] Expliquer pourquoi un meme processus peut avoir plusieurs rangs differents selon le communicateur. / 解释为什么同一进程在不同 communicator 中有不同 rank。

Reponse : Parce que le rang est defini relativement a un communicateur donne. Quand on cree un sous-communicateur, les processus sont renumerotes localement a partir de 0. / 答：因为 rank 只相对于某个 communicator 定义；创建子 communicator 后，进程会在该子组内重新编号。
Exemple complet : voir la question 2 de [APPPartielCorr2024.md](#).

Applications Par Groupes

5.12 [Algorithme:Broadcast pair/impair] Concevoir un programme ou les pairs et impairs diffusent chacun une valeur differente dans leur groupe. / 设计一个程序：奇数组和偶数组各自广播不同的值。

Reponse : On cree d'abord un sous-communicateur pair/impair avec `MPI_Comm_split` . Ensuite, dans chaque groupe, le rang local `0` initialise une valeur differente, puis on appelle `MPI_Bcast` a l'interieur de ce sous-communicateur. / 答：先用 `MPI_Comm_split` 建立奇偶子 communicator；再让每个组的本地 rank 0 初始化不同值，并在各自子 communicator 内调用 `MPI_Bcast` 。

5.13 [Algorithme:Groupes modulo 3] Concevoir un programme ou les groupes rang % 3 effectuent des calculs differents. / 设计一个程序：按 rang % 3 分组后执行不同计算。

Reponse : On utilise `MPI_Comm_split(MPI_COMM_WORLD, rang % 3, rang, &comm3)` . Ensuite on teste la valeur de `rang % 3` pour attribuer un type de calcul a chaque groupe, et on fait les communications collectives a l'interieur de `comm3` . / 答：先用 `MPI_Comm_split(MPI_COMM_WORLD, rang % 3, rang, &comm3)` 按 `rang % 3` 分组，再根据组号让各组执行不同计算，并在 `comm3` 内做通信。

5.14 [Analyse:Isolation des communications] Expliquer comment les communicateurs permettent d'isoler les communications. / 解释 communicator 如何隔离通信。

Reponse : Deux communications portant sur des communicateurs differents n'appartiennent pas au meme espace de communication. Cela evite les interferences entre parties differentes d'une application. / 答：不同 communicator 上的通信属于不同的通信上下文，因此应用中不同部分的通信不会互相干扰。
Exemple complet : voir la question 2 de [APPPartielCorr2024.md](#).

5.15 [Application:Limite de MPI_COMM_WORLD] Donner un cas ou MPI_COMM_WORLD seul ne suffit pas a structurer proprement l'application. / 给出一个仅靠 MPI_COMM_WORLD 不足以清晰组织程序的例子。

Reponse : Par exemple, si les processus pairs doivent collaborer entre eux et les impairs entre eux avec des collectives distinctes, utiliser seulement `MPI_COMM_WORLD` obligerait a ajouter beaucoup de conditions. / 答：例如偶数进程和奇数进程分别要做各自的集合通信时，只用 `MPI_COMM_WORLD` 会导致大量条件判断，不够清晰。

Gestion Et Groupements

5.16 [Impl:Comm_free] Comment detruire un communicateur cree dynamiquement ? / 如何销毁一个动态创建的 communicator?

```
MPI_Comm_free(&new_comm);
```

5.17 [Algorithme:Collectives separees] Donner un exemple ou deux collectives distinctes doivent etre faites dans deux communicateurs differents. / 举一个例子：两个不同 communicator 中需要分别执行各自的集合通信。

Reponse : On peut separer pairs et impairs dans deux communicateurs. Les pairs font un `MPI_Bcast` dans leur groupe, tandis que les impairs font un `MPI_Reduce` dans le leur. / 答：例如把奇偶进程分成两个 communicator；偶数组内部做 `MPI_Bcast` ，奇数组内部做 `MPI_Reduce` 。

5.18 [Algorithme:Blocs de taille 2] Proposer un regroupement des processus par blocs de taille 2 avec MPI_Comm_split . / 用 MPI_Comm_split 按每 2 个进程一组来分组。

```
MPI_Comm bloc2;
MPI_Comm_split(MPI_COMM_WORLD, rang / 2, rang, &bloc2);
```

Ici, 0-1, 2-3, 4-5, ... sont regroupes. / 这里 0-1、2-3、4-5 等会被分到一起。

5.19 [Algorithme:Regroupement modulo 3] Proposer un regroupement des processus selon rang % 3 ./ 按 rang % 3 分组。

```
MPI_Comm comm3;
MPI_Comm_split(MPI_COMM_WORLD, rang % 3, rang, &comm3);
```

Roots Et Rangs Locaux

5.20 [Question:Root local] Dans un groupe pair/impair, comment choisir le root local d'un MPI_Bcast ? / 在奇偶子组里, 如何选择 MPI_Bcast 的本地 root?

Reponse : Le root de MPI_Bcast doit etre un rang local dans le sous-communicateur. On choisit souvent 0, c'est-a-dire le premier rang local de chaque groupe. /

答: MPI_Bcast 的 root 必须是子 communicator 内的本地 rank; 通常选 0, 也就是每组的本地 rank 0。

5.21 [Application:Rangs dans deux communicateurs] Donner un exemple ou le meme processus a des rangs differents dans deux communicateurs distincts. / 举一个例子说明同一个进程在两个 communicator 中可能有不同 rank。

Reponse : Avec 4 processus, le processus de rang global 2 a le rang 2 dans MPI_COMM_WORLD, mais apres un MPI_Comm_split pair/impair, il peut devenir le rang local 1 dans le communicateur des pairs. / 答: 例如 4 个进程时, 全局 rank 2 在 MPI_COMM_WORLD 中 rank 是 2; 若按奇偶分组, 它在偶数组 communicator 中可能变成局部 rank 1。

Duplication Et Construction

5.22 [Definition:Comm_dup] A quoi sert MPI_Comm_dup ? / MPI_Comm_dup 有什么作用?

Reponse : MPI_Comm_dup sert a dupliquer un communicateur en creant un nouveau contexte de communication isole. / 答: MPI_Comm_dup 用于复制一个 communicator, 并创建一个新的、隔离的通信上下文。

5.23 [Definition:Usage de Comm_dup] Citer un cas ou MPI_Comm_dup est preferable a la reutilisation directe de MPI_COMM_WORLD ./ 举一个 MPI_Comm_dup 比直接复用 MPI_COMM_WORLD 更合适的场景。

Reponse : C'est souvent preferable dans une bibliotheque MPI interne, afin d'isoler son propre contexte de communication de celui du reste de l'application. / 答: 在某个内部 MPI 库组件中, 常常更适合用 MPI_Comm_dup, 以便把库自己的通信上下文和应用其他部分隔离开。

5.24 [Definition:Comm_create] A quoi sert MPI_Comm_create dans l'idee generale ? / 从总体思路解释 MPI_Comm_create 有什么作用。

Reponse : Dans l'idee generale, MPI_Comm_create sert a construire un nouveau communicateur a partir d'un sous-ensemble de processus d'un communicateur existant. / 答: 总体上, MPI_Comm_create 用于从已有 communicator 的某个进程子集构造一个新的 communicator。

5.25 [Analyse:Ordre par key] Expliquer pourquoi key ne sert pas seulement a "garder le rang initial", mais a definir l'ordre local dans le nouveau communicateur. / 解释为什么 key 不只是“保留原 rank”, 而是决定新 communicator 内的局部顺序。

Reponse : key est une cle de tri locale. Il ne sert pas uniquement a recopier l'ancien rang ; il permet de choisir comment les nouveaux rangs seront ordonnes dans le sous-communicateur. / 答: key 本质上是组内排序键, 不只是“保留原 rank”; 它决定新 communicator 内局部 rank 的排列顺序。

5.26 [Tableau:Calcul des rangs locaux] Etant donnees des color et key, calculer les nouveaux rangs locaux apres MPI_Comm_split ./ 给定 color 和 key, 计算 MPI_Comm_split 之后的新局部 rank。

Reponse : Il faut d'abord regrouper les processus par color, puis trier chaque groupe par key croissante. Le premier obtient le rang local 0, le suivant 1, etc. En cas d'egalite de key, l'ancien rang departage. / 答: 先按 color 分组, 再在每组内部按 key 升序排序; 排在最前的得到局部 rank 0, 后面依次为 1、2、.....; 若 key 相同, 则由原 rank 打破平局。

5.27 [Analyse:MPI_COMM_NULL] Expliquer ce qu'obtient un processus pour lequel le nouveau communicateur vaut MPI_COMM_NULL ./ 解释当某进程得到 MPI_COMM_NULL 时意味着什么。

Reponse : Cela signifie que ce processus n'appartient a aucun des nouveaux communicateurs construits. Il ne peut donc pas effectuer de communication sur ce communicateur nul. / 答: 这表示该进程没有加入任何新 communicator, 因此不能在这个空 communicator 上再做后续通信。

6. Operations Utilisateur / 6. 用户自定义归约操作

Definitions Des Operations

6.01 [Definition:Op_create] A quoi sert MPI_Op_create ? / MPI_Op_create 有什么作用?

Reponse : MPI_Op_create sert a definir une operation de reduction utilisateur qui pourra ensuite etre utilisee avec des collectives comme MPI_Reduce ou MPI_Allreduce. / 答: MPI_Op_create 用于定义用户自定义归约操作, 之后可与 MPI_Reduce、MPI_Allreduce 等 collective 一起使用。

6.02 [Definition:Prototype utilisateur] Quel prototype doit respecter la fonction utilisateur ? / 用户函数要满足什么原型?

Reponse : La fonction utilisateur doit respecter le prototype suivant : / 答: 用户函数应满足如下原型:

```
void user_op(void *invec, void *inoutvec, int *len, MPI_Datatype *dtype);
```

6.03 [Definition:Associativite] Pourquoi l'operation doit-elle etre associative ? / 为什么操作必须满足结合律?

Reponse : Parce que MPI peut combiner les valeurs dans un ordre arborescent ou variable selon l'implementation. Si l'operation n'est pas associative, le resultat dependrait de l'ordre de combinaison. / 答: 因为 MPI 可能按树形或其他次序组合局部值; 若操作不满足结合律, 最终结果就会依赖组合顺序。

6.04 [Definition:Commute] Que signifie commute ? / commute 表示什么?

Reponse : commute indique si l'operation utilisateur est commutative (1) ou non (0). Une operation commutative laisse plus de liberte d'optimisation a MPI. /

答: commute 表示该用户操作是否满足交换律; 1 表示可交换, 0 表示不可交换。可交换操作通常给 MPI 更多优化空间。

Exemples D Operations

6.05 [Algorithme:Operation d'addition] Decrire une operation utilisateur d'addition. / 描述一个加法用户操作。

Reponse : Le principe consiste a parcourir les len elements et a additionner invec[i] dans inoutvec[i]. Ainsi, inoutvec joue le role d'accumulateur. / 答: 做法是遍历 len 个元素, 把 invec[i] 累加到 inoutvec[i] 中, 因此 inoutvec 扮演累加器角色。

```
void add_op(void *invec, void *inoutvec, int *len, MPI_Datatype *dtype) {
    int *in = (int *)invec;
    int *inout = (int *)inoutvec;
    for (int i = 0; i < *len; i++) inout[i] += in[i];
}
```

6.06 [Algorithme:Maximum absolu] Decrire une operation utilisateur de maximum absolu. / 描述一个绝对值最大用户操作。

Reponse : On compare les valeurs absolues des elements de invec et inoutvec. Si abs(invec[i]) est plus grande, on remplace inoutvec[i] par invec[i]. /

答: 比较 invec 与 inoutvec 对应元素的绝对值; 若 abs(invec[i]) 更大, 就用 invec[i] 替换 inoutvec[i]。

6.07 [Definition:invec/inoutvec] Expliquer le role de invec et inoutvec. / 解释 invec 和 inoutvec 的作用。

Reponse : invec contient les nouvelles valeurs a combiner. inoutvec contient l'accumulateur qui doit etre mis a jour par l'operation utilisateur. / 答: invec 是新输入值, inoutvec 是待更新的累积结果。

6.08 [Definition:Op_free] Expliquer pourquoi il faut liberer l'operation avec MPI_Op_free. / 解释为什么要用 MPI_Op_free 释放操作。

Reponse : Il faut liberer l'operation pour relacher correctement la ressource MPI associee a l'objet MPI_Op. / 答: 需要用 MPI_Op_free 正确释放和该 MPI_Op 对象相关的 MPI 资源。

Applications Et Exercices

6.09 [Algorithme:Somme de carres] Concevoir une reduction utilisateur calculant une somme de carres. / 设计一个平方和用户归约。

Reponse : Chaque processus peut d'abord calculer localement la somme des carres de ses valeurs, puis la reduction utilisateur additionne ces sommes locales. Une autre lecture possible est de definir une operation qui ajoute des contributions deja transformees en carres. / 答: 每个进程可先本地计算平方和, 再用用户归约把这些局部平方和相加; 也可以理解为用户操作组合已转换成平方贡献的值。

6.10 [Algorithme:Maximum pair] Concevoir une reduction utilisateur calculant le maximum des valeurs paires. / 设计一个最大偶数用户归约。

Reponse : L'operation ne garde que les valeurs paires. Si une nouvelle valeur est paire et plus grande que l'accumulateur courant, elle remplace celui-ci ; sinon on ignore cette contribution. / 答：该操作只关注偶数值；若新值是偶数且比当前累积值更大，就更新累积值，否则忽略。

6.11 [Analyse:Interet d'une operation utilisateur] Expliquer quand une operation utilisateur est utile alors que `MPI_SUM` existe deja. / 解释在已有 `MPI_SUM` 时，什么时候自定义操作仍有意义。

Reponse : Une operation utilisateur est utile quand l'operation voulue n'est pas fournie par MPI, par exemple pour combiner plusieurs champs logiques, garder une valeur selon un critere particulier, ou appliquer une regle metier specifique. / 答：当需要的归约规则不是 MPI 已提供的标准操作时，自定义操作就有意义，例如要同时组合多个字段、按特殊准则保留值、或实现某种特定业务规则。

6.12 [Synthese:Op_create + Reduce] Proposer un exercice complet combinant `MPI_Op_create` et `MPI_Reduce`. / 设计一道结合 `MPI_Op_create` 和 `MPI_Reduce` 的完整题。

Reponse : On peut demander de definir une operation utilisateur qui garde la valeur de plus grande norme, puis d'utiliser `MPI_Op_create` pour creer l'operation et `MPI_Reduce` pour obtenir sur la racine la meilleure valeur globale. / 答：例如设计一道题：定义一个保留“范数最大值”的用户操作，用 `MPI_Op_create` 创建后，再用 `MPI_Reduce` 在根进程上得到全局结果。

Proprietes Et Types

6.13 [Algorithme:Minimum absolu] Donner le principe d'une operation utilisateur qui garde le minimum absolu. / 说明一个保留绝对值最小者的用户归约操作原理。

Reponse : On compare `abs(invec[i])` et `abs(inoutvec[i])`. Si la nouvelle valeur a une plus petite valeur absolue, elle remplace l'accumulateur. / 答：比较 `abs(invec[i])` 和 `abs(inoutvec[i])`；若新值的绝对值更小，就用它替换累积值。

6.14 [Analyse:Non-commutativite] Expliquer pourquoi une operation non commutative limite les optimisations possibles. / 解释为什么非交换操作会限制优化。

Reponse : Si l'operation n'est pas commutative, MPI ne peut pas permuter librement l'ordre des contributions. L'implementation dispose donc de moins de liberte pour reorganiser la reduction. / 答：若操作不满足交换律，MPI 就不能自由调换各贡献的组合顺序，因此可用的优化空间更小。

6.15 [Algorithme:Reduction sur structure] Donner un exemple de reduction utilisateur qui combine deux champs d'une structure logique. / 给出一个对逻辑结构中两个字段同时归约的用户操作例子。

Reponse : On peut imaginer une structure logique avec deux champs, par exemple `somme` et `compte`. L'operation additionne les deux champs composante par composante, ce qui permet ensuite de calculer une moyenne globale. / 答：例如可定义一个含 `somme` 和 `compte` 两个字段的逻辑结构；用户操作按分量分别相加，最后即可据此算全局平均值。

6.16 [Analyse:Coherence datatype/buffer] Expliquer pourquoi il faut etre coherent entre `datatype` MPI et le contenu reel des buffers. / 解释为什么 `datatype` 必须和缓冲区实际内容保持一致。

Reponse : Parce que l'operation utilisateur interprete les donnees selon le type logique attendu. Si le `datatype` MPI ne correspond pas au vrai contenu des buffers, l'interpretation memoire devient fausse et le resultat est incorrect. / 答：因为用户操作会按预期逻辑类型解释数据；若 MPI `datatype` 与缓冲区真实内容不一致，就会造成错误解释，结果也会错误。

Booleens Et Comptages

6.17 [Algorithme:Comptage des positifs] Proposer une reduction utilisateur qui compte combien de valeurs sont strictement positives. / 设计一个统计正数个数的用户归约。

Reponse : Chaque processus peut compter localement combien de valeurs sont strictement positives, puis l'operation utilisateur additionne ces compteurs. / 答：每个进程先本地统计正数个数，再由用户操作把这些局部计数相加。

6.18 [Algorithme:ET logique] Proposer une reduction utilisateur qui combine des booleens par ET logique. / 设计一个把布尔值做逻辑与的用户归约。

Reponse : L'operation prend deux booleens et remplace l'accumulateur par `accumulateur && nouvelle_valeur`. / 答：该操作把当前累积布尔值和新布尔值做逻辑与，即更新为 `acc && val`。

6.19 [Algorithme:OU logique] Proposer une reduction utilisateur qui combine des booleens par OU logique. / 设计一个把布尔值做逻辑或的用户归约。

Reponse : L'operation prend deux booleens et remplace l'accumulateur par `accumulateur || nouvelle_valeur`. / 答：该操作把当前累积布尔值和新布尔值做逻辑或，即更新为 `acc || val`。

7. Types Derives MPI / 7. MPI 派生类型

Definitions Des Types Derives

7.01 [Definition:Type derive] Qu'est-ce qu'un type derive MPI ? / 什么是 MPI 派生类型？

Reponse : Un type derive MPI est un type construit a partir de types MPI existants pour deccrire une organisation memoire plus complexe qu'un simple bloc contigu. / 答：MPI 派生类型是基于已有 MPI 类型构造出来的类型，用于描述比简单连续块更复杂的内存布局。

7.02 [Definition:Utilite des types derives] Pourquoi les types derives sont-ils utiles ? / 为什么派生类型有用？

Reponse : Ils permettent de communiquer directement des donnees non contigues ou structurees sans recopier manuellement les elements dans un tampon temporaire. / 答：派生类型可以直接通信非连续或结构化数据，而不必先手工复制到临时缓冲区。

7.03 [Definition:Type_commit] A quoi sert `MPI_Type_commit` ? / `MPI_Type_commit` 有什么作用？

Reponse : `MPI_Type_commit` sert a valider un type derive avant son utilisation dans une communication MPI. / 答：`MPI_Type_commit` 用于在通信前正式提交并激活一个派生类型。

7.04 [Definition:Type_free] A quoi sert `MPI_Type_free` ? / `MPI_Type_free` 有什么作用？

Reponse : `MPI_Type_free` sert a liberer la ressource MPI associee a un type derive lorsque l'on n'en a plus besoin. / 答：`MPI_Type_free` 用于释放派生类型占用的 MPI 资源。

7.05 [Definition:Type_contiguous] Expliquer `MPI_Type_contiguous`. / 解释 `MPI_Type_contiguous`。

Reponse : `MPI_Type_contiguous` cree un nouveau type correspondant a plusieurs elements consecutifs d'un type de base. / 答：`MPI_Type_contiguous` 用于把若干个连续的基础元素组合成一个新类型。

```
MPI_Datatype t4int;
MPI_Type_contiguous(4, MPI_INT, &t4int);
MPI_Type_commit(&t4int);
```

Exemple complet : voir 7.15.

7.06 [Definition:Type_vector] Expliquer `MPI_Type_vector`. / 解释 `MPI_Type_vector`。

Reponse : `MPI_Type_vector` cree un type compose de plusieurs blocs de meme taille separes par un pas constant exprime en nombre d'elements. / 答：`MPI_Type_vector` 用于构造“多个等长块 + 固定步长”的派生类型，步长单位是元素个数。

7.07 [Definition:Type_create_hvector] Expliquer `MPI_Type_create_hvector`. / 解释 `MPI_Type_create_hvector`。

Reponse : `MPI_Type_create_hvector` ressemble a `MPI_Type_vector`, mais le pas est exprime en octets et non en nombre d'elements. / 答：`MPI_Type_create_hvector` 与 `MPI_Type_vector` 类似，但步长单位是字节而不是元素个数。

Comparaisons Et Motifs Reguliers

7.08 [Comparaison:Stride elements vs octets] Comparer stride en elements et stride en octets. / 比较“按元素步长”和“按字节步长”。

Reponse : Un stride en elements compte combien d'elements de base separant deux blocs. Un stride en octets exprime directement l'ecart memoire en bytes. / 答：按元素步长表示两个块之间相隔多少个基础元素；按字节步长则直接表示内存地址间隔多少字节。

7.09 [Application:Un element sur deux] Donner un exemple pour envoyer un element sur deux d'un tableau. / 举例说明如何发送数组中隔一个取一个的元素。

Reponse : On peut utiliser `MPI_Type_vector` avec des blocs de longueur 1 et un stride de 2. / 答：可以使用块长度为 1、步长为 2 的 `MPI_Type_vector`。

```
MPI_Datatype un_sur_deux;
MPI_Type_vector(n, 1, 2, MPI_INT, &un_sur_deux);
MPI_Type_commit(&un_sur_deux);
```

Colonnes Et Applications

7.10 [Application:Colonne de matrice] Donner un exemple pour envoyer une colonne de matrice. / 举例说明如何发送矩阵的一列。

Reponse : Si la matrice est stockee ligne par ligne, une colonne peut etre decrite par un `MPI_Type_vector` avec `nombreBloc = nb_lignes`, `longueurBloc = 1` et `stride = nb_colonnes`. / 答：如果矩阵按行存储，一整列可以用 `MPI_Type_vector` 描述：块数是行数，块长度是 1，步长是列数。

7.11 [Synthese:Exercice colonne] Concevoir un exercice complet ou l'on envoie une colonne de matrice par MPI. / 设计一道完整题：用 MPI 发送矩阵的一列。
Reponse : On peut demander de definir un type colonne avec `MPI_Type_vector`, de le valider avec `MPI_Type_commit`, puis de l'utiliser dans un `MPI_Send` / `MPI_Recv` entre deux processus, avant de le liberer avec `MPI_Type_free`. / 答：可以设计成：先用 `MPI_Type_vector` 定义列类型，再 `MPI_Type_commit`，随后在两个进程之间用 `MPI_Send/MPI_Recv` 发送该列，最后 `MPI_Type_free`。

7.12 [Comparaison:Type derive vs copie manuelle] Comparer type derive et copie manuelle dans un devoir. / 在题目里比较派生类型和手工复制。
Reponse : La copie manuelle est souvent plus simple a imaginer au debut, mais elle allonge le code et multiplie les boucles auxiliaires. Le type derive exprime directement la structure logique des donnees et rend l'intention plus claire. / 答：手工复制一开始更直观，但代码更长、辅助循环更多；派生类型更直接表达数据布局，代码意图也更清晰。

7.13 [Analyse:Clarte et portabilite] Expliquer l'interet des types derives en clarte et en portabilite. / 解释派生类型在代码清晰度和可移植性上的好处。
Reponse : Ils rendent explicite la forme memoire des donnees et evitent des manipulations ad hoc. En laissant MPI gerer la description du layout, on obtient aussi une solution plus portable. / 答：派生类型把数据布局显式描述出来，避免手工拼接数据；让 MPI 负责处理 layout，也通常更具可移植性。

Exemple complet : voir 7.14 .

7.14 [Impl:Type_vector + collective] Proposer un programme utilisant a la fois `MPI_Type_vector` et une collective. / 设计一个同时使用 `MPI_Type_vector` 和集合通信的程序。

Reponse : On peut imaginer que chaque processus possede une matrice locale et utilise un type colonne construit par `MPI_Type_vector`, puis participe a un `MPI_Gather` de colonnes vers la racine. / 答：例如每个进程持有一个局部矩阵，用 `MPI_Type_vector` 定义“列类型”，然后通过 `MPI_Gather` 把各列收集到根进程。

Types Simples Et Lignes

7.15 [Impl:4 int contigus] Proposer la creation d'un type correspondant a 4 `int` consecutifs. / 设计一个表示 4 个连续 `int` 的派生类型。

```
MPI_Datatype t4int;
MPI_Type_contiguous(4, MPI_INT, &t4int);
MPI_Type_commit(&t4int);
```

7.16 [Impl:Ligne de matrice] Proposer la creation d'un type correspondant a une ligne de matrice. / 设计一个表示矩阵一行的派生类型。

Reponse :

```
MPI_Datatype ligne;
MPI_Type_contiguous(nb_colonnes, MPI_DOUBLE, &ligne);
MPI_Type_commit(&ligne);
```

7.17 [Analyse:Interet de Type_vector] Expliquer dans quel cas `MPI_Type_vector` est preferable a une boucle de copies. / 解释在什么情况下 `MPI_Type_vector` 比手工复制更合适。

Reponse : `MPI_Type_vector` est preferable quand la donnee suit un motif regulier non contigu, comme une colonne ou des elements a pas constant. / 答：当数据是“规则的非连续布局”时，例如矩阵列或固定步长抽样，`MPI_Type_vector` 通常比手工复制更合适。

Structures Et Layouts Irreguliers

7.18 [Analyse:Alignement memoire] Expliquer pourquoi l'alignement memoire peut etre important dans les types structures. / 解释为什么结构类型中内存对齐很重要。

Reponse : Dans une structure, les champs peuvent etre separes par du padding ajoute pour l'alignement. Si ce padding n'est pas pris en compte, la description MPI du layout devient fausse. / 答：结构体中的字段之间可能因为对齐而插入 padding；若忽略这些 padding，MPI 对内存布局的描述就会出错。

7.19 [Application:Blocs reguliers] Donner un exemple ou l'on veut envoyer 3 blocs de 2 elements separes par un pas constant. / 举例说明如何发送 3 个长度为 2、步长固定的块。

Reponse : On utilise `MPI_Type_vector(3, 2, stride, MPI_INT, &t)` avec un `stride` adapte au tableau concerne. / 答：可以使用

`MPI_Type_vector(3, 2, stride, MPI_INT, &t)`，其中 `stride` 取决于实际数组布局。

7.20 [Application:Type derive en point-a-point] Donner un exemple de communication point-a-point utilisant un type derive. / 给出一个使用派生类型的点对点通信例子。

Reponse : Un exemple classique est l'envoi d'une colonne de matrice d'un processus 0 vers un processus 1 avec `MPI_Send` et un type colonne derive. / 答：典型例子是用派生出的“列类型”，通过 `MPI_Send` 把矩阵的一列从进程 0 发给进程 1。

7.21 [Application:Type derive en collective] Donner un exemple de communication collective utilisant un type derive. / 给出一个使用派生类型的集合通信例子。
Reponse : On peut utiliser un type derive colonne avec `MPI_Bcast` ou `MPI_Gather` pour diffuser ou rassembler directement des colonnes sans copie manuelle. / 答：例如可把“列类型”与 `MPI_Bcast` 或 `MPI_Gather` 搭配使用，直接广播或收集矩阵列而无需手工复制。

Struct, Indexed Et Extent

7.22 [Definition:Type_create_struct] Expliquer a quoi sert `MPI_Type_create_struct`. / 解释 `MPI_Type_create_struct` 的作用。

Reponse : `MPI_Type_create_struct` sert a decrire un type compose de plusieurs champs de types differents, avec des deplacements explicites en memoire. /

答： `MPI_Type_create_struct` 用于描述由多个不同类型字段组成的结构体类型，并显式指定它们在内存中的位移。

7.23 [Definition:Type_indexed/indexed] Expliquer a quoi servent `MPI_Type_indexed` et `MPI_Type_create_hindexed`. / 解释 `MPI_Type_indexed` 和 `MPI_Type_create_hindexed` 的作用。

Reponse : Ces fonctions servent a decrire des blocs de tailles variables separes par des deplacements variables. `indexed` exprime ces deplacements en elements, `hindexed` en octets. / 答：这两个函数用于描述“块长度可变、位移也可变”的布局；`indexed` 的位移单位是元素，`hindexed` 的位移单位是字节。

7.24 [Comparaison:Type_vector vs Type_indexed] Expliquer la difference entre `MPI_Type_vector` et `MPI_Type_indexed`. / 解释 `MPI_Type_vector` 与 `MPI_Type_indexed` 的区别。

Reponse : `MPI_Type_vector` suppose des blocs tous de meme taille et un stride constant. `MPI_Type_indexed` permet au contraire des tailles et deplacements plus flexibles. / 答： `MPI_Type_vector` 适合“块大小一致、步长固定”的规则布局；`MPI_Type_indexed` 则更灵活，支持不规则块大小和位移。

7.25 [Comparaison:Hvector vs Vector] Expliquer dans quel cas `MPI_Type_create_hvector` est preferable a `MPI_Type_vector`. / 解释在什么情况下 `MPI_Type_create_hvector` 比 `MPI_Type_vector` 更合适。

Reponse : `MPI_Type_create_hvector` est preferable quand le stride doit etre exprime en bytes, par exemple lorsqu'on manipule deja des offsets memoire explicites ou des types non triviaux. / 答：当步长必须以字节表示时，`MPI_Type_create_hvector` 更合适，例如已经直接操作字节级内存位移或复杂类型时。

7.26 [Application:Structure C] Proposer un type derive correspondant a une structure C avec deux champs de types differents. / 为一个含两种不同字段类型的 C 结构体设计派生类型。

Reponse : On peut prendre une structure `struct { int i; double x; }`, puis utiliser `MPI_Type_create_struct` avec deux blocs, les types `MPI_INT` et `MPI_DOUBLE`, et les deplacements calcules proprement. / 答：例如对 `struct { int i; double x; }`，可用 `MPI_Type_create_struct` 定义两个块，对应 `MPI_INT` 和 `MPI_DOUBLE`，并提供正确位移。

7.28 [Application:Cas naturel pour Type_indexed] Donner un exemple ou `MPI_Type_indexed` est plus naturel que `MPI_Type_vector`. / 给出一个 `MPI_Type_indexed` 比 `MPI_Type_vector` 更自然的例子。

Reponse : Si l'on veut envoyer les elements `a[0]`, `a[3]`, `a[4]`, `a[10]`, la disposition n'est pas reguliere. `MPI_Type_indexed` est alors plus naturel que `MPI_Type_vector`. / 答：例如要发送 `a[0]`、`a[3]`、`a[4]`、`a[10]` 这样的不规则位置元素时，`MPI_Type_indexed` 比 `MPI_Type_vector` 更自然。

7.29 [Definition:Type_size/Type_get_extent] Expliquer a quoi servent `MPI_Type_size` et `MPI_Type_get_extent`. / 解释 `MPI_Type_size` 和 `MPI_Type_get_extent` 的作用。

Reponse : `MPI_Type_size` donne la taille logique des donnees utiles d'un type. `MPI_Type_get_extent` donne son etendue memoire, c'est-a-dire l'intervalle memoire couvre par ce type. / 答： `MPI_Type_size` 给出类型中有效数据的逻辑大小；`MPI_Type_get_extent` 给出该类型在内存中的跨度范围。

7.30 [Analyse:Taille logique vs extent] Expliquer pourquoi la taille logique d'un type et son extent ne sont pas toujours identiques. / 解释为什么一个类型的逻辑大小和 extent 不总是相同。

Reponse : Parce qu'un type peut contenir des trous memoire dus au stride, au padding ou a l'alignement. La quantite de donnees utiles et l'espace memoire couvre ne sont donc pas toujours egaux. / 答：因为派生类型可能包含由步长、padding 或对齐带来的内存空洞，所以“有效数据大小”和“内存跨度”不一定相同。